04820KA001 Mono Canada II_Ital_Mod 1026_1

Author: Michou
Created: 10/26/2026 2:56:15 PM
Last Modified: 10/26/2026 2:56:15 PM
Comment: Modified to Connect 2nd Diameter Sens

Sysmac Studio Module Version: 1.60.0.64010

1.PLC

1-7.POUs

1-7-1.Programs

1-7-1-1.TaglMonoTRM

1-7-1-1-1.Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Comment
VAR						
SpdDelta	REAL			False	False	Minimum or maximum speed SpdDeltaerence
SpdDeltaMin	REAL			False	False	Minimum speed difference
SpdDeltaMax	REAL			False	False	Maximum speed SpdDeltaerence
Clock100ms	BOOL			False	False	100 ms clock
MemPos	DINT			False	False	Previous encoder count
DeltaPulse	DINT			False	False	Encoder count difference
SlowMem	BOOL			False	False	SlowDown edge memory
DecelDiam	REAL			False	False	Rewinder diameter used by the decel model (mm, min 100)
TeachSwap	REAL			False	False	Temporary for UNDO swap
TESTBOOL	bool			False	False	
SlowStartDiam	REAL			False	False	Diameter at slow-down start (mm)
BrakeHold	BOOL			False	False	Keep T-one in auto during a normal stop (O_Brake)
StopNow	BOOL			False	False	Immediate stop request (replaces I_Stop_PB in General rung 9)
StopPB_Mem	BOOL			False	False	STOP button edge memory
T_StopLong	TON			False	False	STOP held 1 s = immediate stop
T_SoftStop	TON			False	False	Soft stop timeout (120 s)
BrakeHoldTicks	INT			False	False	BrakeHold duration after stop (100 ms ticks, max 150)
StopStartDiam	REAL			False	False	Diameter at stop command (mm)
DecelK_Meas	REAL			False	False	Measured decel per mm of diameter
SlowMeasActive	BOOL			False	False	Slow-down measurement running
TeachTicks	INT			False	False	TEACH CORE stable time (100 ms ticks)
SlowStartPos	DINT			False	False	Encoder at slow-down start
SlowStartSpd_Pps	REAL			False	False	Speed at slow-down start (pulses/s)
SlowMeas_Pls	REAL			False	False	Measured slow-down length (pulses)
SlowDecelMeas_mm	REAL			False	False	Measured slow-down decel (mm/s2)
SlowOneRamp	BOOL			False	False	Slow-down uses one ramp (manual or learned)
Velocity	REAL			False	False	Speed in [mm/s]
Alarm_Array	ARRAY[0..32] OF BOOL			False	False	Alarm
T_PlcOn	TON			False	False	PLC active timeout
ControlVoltage	BOOL			False	False	Auxiliaries OK
StartMotor_Cmd	BOOL			False	False	Motor start command
HubExtractEnable	BOOL			False	False	Hub extraction enable command
Brake_Cmd	BOOL			False	False	Brake command
T_StopVent	TOF			False	False	Fans stop timer
TimStopVen	TIME			False	False	Fans stop timeout

MotorCooling	BOOL			False	False	Fans command
OverRider_Auto	BOOL			False	False	OverRider in automatic
OverRider_Cont	BOOL			False	False	OverRider in continuous
OverRider_Lock	BOOL			False	False	OverRider lock command
GSM_LearnFault	BOOL			False	False	Paper [Grams / Square Meter] LearnFailCnt ≥ RwLearnFailMax
TickCnt	INT			False	False	100 ms tick scan counter (25 x 4 ms)
RunWasJog	BOOL			False	False	Run used jog / manual / slow button (2nd ramp)
StopLearnValid	BOOL			False	False	This stop may be learned
SpdBand_Pps	REAL			False	False	Speed band of the trim ramp (pulses/s)
TrimDecel_Pps2	REAL			False	False	Trim ramp decel (pulses/s2)
SpdKnee_Pps	REAL			False	False	Speed where coarse ramp hands over to trim ramp (pulses/s)
OverRider_RaiseUp	BOOL			False	False	OverRider raise command
Pot1_Mem	INT			False	False	Work potentiometer value
Clock10ms	BOOL			False	False	10 ms clock
VelSet_mms	REAL			False	False	Set speed in mm/s
Vfd_SP_Modif	REAL			False	False	Inverter speed modifier
UwTeachTicks	INT			False	False	Unwinder TEACH CORE samples (100 ms)
T_PosPresOff	TON			False	False	OverRider photocell OFF timer
T_PosPresOn	TON			False	False	OverRider photocell ON timer
TimPosPresOff	TIME			False	False	OverRider photocell OFF timeout
TimPosPresOn	TIME			False	False	OverRider photocell ON timeout
RollEnd_Active	BOOL			False	False	Roll diameter photocell intercepted
Output_Ratio	REAL			False	False	Analog output ratio
Input_Ratio	REAL			False	False	Analog input ratio
T_StartLow	TON			False	False	Slow start timer
T_UwDiamDebounce	TON			False	False	Roll diameter photocell Debounce timer 500mS
UwDiamReached	BOOL			False	False	Roll diameter reached
Index_A	INT			False	False	
AlarmStrobe	BOOL			False	False	Alarm message display strobe
T_Display	TON			False	False	Alarm display timer
RangeOK	BOOL			False	False	Range reached
Vel_Max_mms	REAL			False	False	Maximum speed in mm/s
Slow_Inhibit	BOOL			False	False	Inhibit slow button
Decel_Ramp	REAL			False	False	Deceleration ramp
Accel_Ramp	REAL			False	False	Acceleration ramp
Pot2_Mem	INT			False	False	Final potentiometer value
Current_mm	REAL			False	False	Current millimetres
EncoderPulse_to_mm	REAL			False	False	mm per encoder pulse
SlowDown	BOOL			False	False	Slow down
Stop	BOOL			False	False	Stop
T_EncoderStill	TON			False	False	(?) Encoder stopped timer
Analog_Res	UINT			True	False	Analog board resolution
Mt_To_mm	REAL			False	False	m/min to mm/s factor
Sec_to_Min	REAL			False	False	mm/s to m/min factor
Pot1_Word	WORD			False	False	(?) Pot 1 raw word
Pot1_WordSW	WORD			False	False	(?) Pot 1 raw word, byte-swapped
Pot2_Word	WORD			False	False	(?) Pot 2 raw word

Pot2_WordSW	WORD			False	False	(?) Pot 2 raw word, byte-swapped
InvRef2	UINT			False	False	(?) Inverter reference 2
InvRefSW	WORD			False	False	(?) Inverter reference, byte-swapped
InvRefWW	WORD			False	False	(?) Inverter reference, word
Init_Ramp	BOOL			True	False	(?) Ramp initialisation
SlowByDiam	BOOL			False	False	(?) Slow down by diameter
UwStopByDiam	BOOL			False	False	Stop by diameter
T_Screen_Delay	TON			False	False	(?) Start page timer
T_PulStop	TON			False	False	(?) Stop button timer
Reading_Max	REAL	2000		True	False	Maximum sensor reading (full scale)
DeadZone	REAL	60		True	False	Sensor dead zone
In_UwDiameter	REAL			True	False	UnWinder Diameter Analog input voltage
T_Sample	TON			False	False	100 ms tick timer
Tick	BOOL			False	False	100 ms tick
MmPerPulse	REAL			False	False	mm per encoder pulse
RwDistRaw_mm	REAL			False	False	Rewinder Unfiltered distance (mm)
RwFiltInit	BOOL			False	False	Rewinder Filter initialised
SpdInit	BOOL			False	False	Speed init
EncPrev	DINT			False	False	Encoder at previous tick
SpdAct_Pps	REAL			False	False	Actual Speed (pulses/s)
LearnInit	BOOL			False	False	Learn initialised
LearnWinRunning	BOOL			False	False	Motor ran whole window
LearnWinTicks	INT			False	False	Learn window counter
Encoder_Last	DINT			False	False	Encoder at window start
Diameter_Last	REAL			False	False	Diameter at window start
LearnWin_Pls	DINT			False	False	Pulses in window
LearnWinDiamDelta	REAL			False	False	Diameter change in window
MatK_Window	REAL			False	False	Material factor this window
MotorRunMem	BOOL			False	False	Motor start edge memory
StopMeasActive	BOOL			False	False	Stop measurement running
StopStartPos	DINT			False	False	Encoder at stop command
StopStartSpd_Pps	REAL			False	False	Speed at stop command (pulses/s)
StandstillTicks	INT			False	False	Standstill tick counter
LearnWinOk	BOOL			False	False	Learning window gave a valid result
ThickWin_mm	REAL			False	False	Paper thickness this window (mm)
LearnFailCnt	INT			False	False	Consecutive failed learning windows
StopLearnOk	BOOL			False	False	Stop measurement gave a valid result
StopDecelMeas_mm	REAL			False	False	Measured stop decel (mm/s2)
StopMeas_Pls	REAL			False	False	Measured stop length (pulses)
StopDecelMeas	REAL			False	False	Measured stop decel (pulses/s2)
StopLen_Pls	REAL			False	False	Stop length (pulses)
StopLenFinal_Pls	REAL			False	False	Stop length from slow speed (pulses)
RwSlowLen_Pls	REAL			False	False	Rewinder Slow-down length (pulses)
SlowDecel_Pps2	REAL			False	False	Slow-down decel (pulses/s2)
SpdFinal_Pps	REAL			False	False	Final speed (pulses/s)

	RwRemain_Pls	REAL			False	False	Rewinder Remaining length to target (pulses)
	LearnWinStartPos	DINT			False	False	Rewinder Position Window 0
	LearnWinStartDiam	REAL			False	False	Rewinder Diameter Window 0
	T_ByPassProtOff	TOF			False	False	Delay Off Timer
	UwEnMem	BOOL			False	False	Edge memory UwStopEnable
	RwEnMem	BOOL			False	False	Edge memory RwEnable
	SpdRef_Pps	REAL			False	False	Reference speed MAX (actual,setpoint) (pulses/s)
	SpdWork_Pps	REAL			False	False	Work speed setpoint (pulses/s)
	SlowLenBase_Pls	REAL			False	False	Slow-down length without margin (pulses)
	UwSlowLen_Pls	REAL			False	False	Unwinder slow-down length (pulses)
	UwMarginUsed_m	REAL			False	False	Unwinder margin at slow speed (m)
	SqrtArg	REAL			False	False	Square root argument
	AutoSlowCmd	BOOL			False	False	Automatic Slow Command
	AutoStopCmd	BOOL			False	False	Automatic Stop Command
	RwLastValidDiam	REAL			False	False	Rewinder Last valid diameter (mm)
	RwRemainMax_Pls	REAL			False	False	Rewinder Remaining length to max diameter (pulses)
	UnWinder_Empty	BOOL			False	False	UnWinder Empty
	StopFailCnt	INT			False	False	Consecutive failed stop measurements
	UwTeachSum	REAL			False	False	Unwinder TEACH CORE sonar sum (mm)
VAR_EXTERNAL							
	VelSet	REAL				False	Set speed
	OverSpeed	BOOL				False	Speed too high
	SpdDelta_Percent	REAL				False	Speed difference percentage
	Encoder_Pos	DINT				False	Encoder counter
	Velocity_M_min	REAL				False	Speed in m/min
	Current_Metre	REAL				False	Current metres
	I_RsetCnt	BOOL				False	Metre counter reset button
	WorkSpd_Max_Set	REAL				False	HMI request: work pot full scale (m/min), applied only with motor stopped
	FinalSpd_Max_Set	REAL				False	HMI request: final pot full scale (m/min), applied only with motor stopped
	SpdMax_Pending	BOOL				False	New pot range waiting for the motor to stop
	RwAxisDist_Prev	REAL				False	Rewinder: RwAxisDist before the last TEACH (for UNDO)
	UwCoreDist_Prev	REAL				False	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
	RwTeachUndo	BOOL				False	HMI: UNDO rewinder TEACH (PLC resets it)
	UwTeachUndo	BOOL				False	HMI: UNDO unwinder TEACH (PLC resets it)
	UnderSpeed	BOOL				False	Speed too low
	I_ControlVoltage	BOOL				False	Auxiliaries ON
	I_OverRider_PH	BOOL				False	Roll diameter photocell

RwCoreOD	REAL				False	Rewinder core outside diameter for TEACH CORE (mm)
UwTeachState	INT				False	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SoftStop	BOOL				False	Soft stop in progress (HMI)
SoftStopEnable	BOOL				False	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL				False	Soft stop: run command drops below this speed (m/min)
SoftStopDecel	REAL				False	Soft stop decel (mm/s2), 0 = DecelCoarse
RwTeachCore	BOOL				False	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT				False	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
O_MotorStart	BOOL				False	Motor start
RwSlowDecelK	REAL				False	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwStopDecelK	REAL				False	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
I_OR_Position_Sens	BOOL				False	Rider roll position sensor
WorkSpd_Max	REAL				False	Work speed pot full scale (m/min), 50..VelMax
FinalSpd_Max	REAL				False	Final speed pot full scale (m/min), 5..WorkSpd_Max
I_OR_Pressure_Sens	BOOL				False	Rider roll pressure switch
I_OR_Auto_OS	BOOL				False	Rider roll in automatic
I_OR_Cont_OS	BOOL				False	Rider roll in continuous
I_Start_PB	BOOL				False	Start button
I_SafeGuard_Closed	BOOL				False	Side doors closed
I_Motor_Therma l	BOOL				False	Motor thermal switch
I_FanTemp_Ok	BOOL				False	Motor (fan) thermal OK
I_Stop_PB	BOOL				False	Stop button
I_Hub_Position	BOOL				False	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL				False	Front guard closed
RwMaxSlowFallback	REAL				False	Rewinder max: slow-down diameter while K not learned (mm)
RwMaxSlowAtDiam	REAL				False	Rewinder max: slow-down starts at diameter (mm)
O_Alarm_Array	BOOL				False	Alarm lamp
O_RwUnload	BOOL				False	Unloaders enable - opening
I_Prot_ByPass	BOOL				False	Guards-bypassed selector switch
O_Prot_ByPass_Lamp	BOOL				False	Guards bypassed lamp
O_Brake	BOOL				False	Brake command
Fan_Timeout	REAL				False	Fans stop timeout (from panel)
O_CoolingFan	BOOL				False	Brake and motor fans
O_OR_Auto_Lamp	BOOL				False	Rider roll auto/continuous lamp
O_OverRider_Up	BOOL				False	Raise rider roll
O_OverRider_Hold	BOOL				False	Lock rider roll

Fix10Vdc	INT				False	Fixed 10 Vdc generator
InvRef	INT				False	Inverter speed reference
Encoder_On	BOOL				False	Encoder enable
Encoder_Reset	BOOL				False	Encoder reset
OverRider_Time OFF	REAL				False	Rider roll photocell OFF timeout (panel)
OverRider_Time ON	REAL				False	Rider roll photocell ON timeout (panel)
VelMax	REAL				False	Maximum speed
I_LowSpd_OS	BOOL				False	Slow speed button
O_Motor_Lamp	BOOL				False	Motor ON lamp
AlarmOn	BOOL				False	Alarm active
BuzzerOn	BOOL				False	Panel buzzer command
I_VFD_Alarm	BOOL				False	Inverter in alarm
Msg	INT				False	Message
Index_B	INT				False	Active alarm index
MotStop	BOOL				False	Motor stopped
Clock1s	BOOL				False	1 s clock
ManMode	BOOL				False	Manual mode
Speed_Man_SP	INT				False	% speed in manual
Counter_Reset	BOOL				False	Panel counter reset button
I_FrontGuard_Open	BOOL				False	Front guard open
RwUnload_Enable	BOOL				False	Unloaders enable - closing
JogSpd_SP	REAL				False	Speed in manual
O_AccDec2	BOOL				False	Second acc/dec ramp
AccelTrim	REAL				False	Acceleration coefficient in run
DecelTrim	REAL				False	Deceleration coefficient in run
DecelCoarse	REAL				False	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL				False	Deceleration coefficient at start (orig. comment; name suggests accel)
Speed_Display	REAL				False	Set speed (displayed)
O_AcousticSignal	BOOL				False	Horn / acoustic signal
Pot1_Work_Velocity	INT				False	Potentiometer 1 value - Work speed -
O_FrontGuard_ByPass	BOOL				False	Front guard bypass
Pot2_Slow_Velocity	INT				False	Potentiometer 2 value - Final speed -
LowSpd_Display	REAL				False	Slow-down speed (displayed)
Encoder_Pulse	INT				False	Encoder pulses
LowSpd_Man_SP	INT				False	% slow-down speed in manual
RwSlowDecelLrn	REAL				False	Learned real slow-down decel (mm/s2) - 0 = not learned
UwSlowMargin_m	REAL				False	Slow-down metres
UwStop_At_Meter	REAL				False	Total metres
I_AutoStop_OS	BOOL				False	Stop on metre counter
I_RollEnd_SlowDown	BOOL				False	Slow down at end of roll
I_SlowSens_PH	BOOL				False	Slow-down photocell
I_WebBreak_Monitor	BOOL				False	Paper break monitoring ON

I_WebBreak_PH	BOOL				False	Paper break photocell
I_Vacuum_OK	BOOL				False	Extractor fan thermal OK
CountReset_Req	BOOL				False	Metre counter reset request
Psw	INT				False	Password
I_RsetAlarm	BOOL				False	Alarm reset
Fix10Vdc_1	UINT				False	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL				False	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL				False	Panel alarms
SafeGuard_ByPass	BOOL				False	Guards bypassed
FrontGuard_Closed	BOOL				False	Front guard closed
Core_Measure	BOOL				False	(?) Stored core value
UwDiam_Measure	REAL				False	Measured distance
UwCoreDistance	REAL				False	(?) Core measurement
AI_Diam_Unwinder	INT				False	Analog value - parent roll measurement
UwCoreDiameter	REAL				False	(?) Shaft diameter
Measure_ON	BOOL				False	(?) Measurement enabled
UwDiam_Act	REAL				False	(?) Measured roll diameter
UwDiamStop	REAL				False	(?) Stop diameter
UwSlowDown_Diam	REAL				False	(?) Slow-down diameter
Alarm_Reset	BOOL				False	(?) Alarm reset from panel
HMI_Screen	ARRAY[0..1] OF DINT				False	HMI panel page
Psw22	BOOL				False	(?) Password 22
Psw44	BOOL				False	(?) Password 44
I_Roll_OK	BOOL				False	(?) Roll OK
I_RollEnd_PH	BOOL				False	(?) End-of-roll photocell
AI_Diam_Rewinder	INT				False	Rewinder DT35 raw - I/O Map Unit2 Ch2
RwStopEnable	BOOL				False	Rewinder diameter stop enabled
RwTargetDiam	REAL				False	Rewinder final diameter (mm)
RwUseSlow	BOOL				False	Slow down before target
RwSlowMargin_m	REAL				False	Metres at slow speed before stop
RwSlowDecel	REAL				False	Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL				False	Reset requests below target - this (mm)
RwDist0V	REAL				False	DT35 distance at 0 V (mm)
RwDist10V	REAL				False	DT35 distance at 10 V (mm)
RwRawMin	INT				False	Raw below = sensor fault
RwRawMax	INT				False	Raw above = sensor fault
RwStopOnFault	BOOL				False	Stop on sensor fault
RwAxisDist	REAL				False	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL				False	Distance filter per 100 ms (0..1)
RwWinTicks	INT				False	Material learn window (100 = 10 s)
RwMinPulses	DINT				False	Min pulses in window to learn
RwLearn_K	REAL				False	Material factor filter weight
RwMat_K	REAL				False	Learned material factor
RwLearnStop	BOOL				False	Learn stop decel on every stop
RwMinLearnSpd	REAL				False	Min speed to learn a stop (m/min

						)
RwStopDecel	REAL				False	Learned stop decel (pulses/s2)
RwTreact	REAL				False	Stop reaction time (s)
RwTstopDef	REAL				False	Stop time until decel learned (s)
RwTrim	REAL				False	Stop length trim (>1 = earlier)
RwDist	REAL				False	Filtered distance (mm)
RwDiam	REAL				False	Rewinder diameter (mm)
RwSpeed	REAL				False	Line speed from encoder (m/min)
RwRemain_m	REAL				False	Metres to target diameter
RwStopLen_m	REAL				False	Stopping length now (m)
RwSlowLen_m	REAL				False	Slow-down length now (m)
RwStopReq	BOOL				False	Rewinder diameter STOP request
RwSlowReq	BOOL				False	Rewinder diameter SLOW request
RwSensFault	BOOL				False	Rewinder DT35 sensor fault
UwStopEnable	BOOL				False	Unwinder metre-counter stop ON (HMI)
SlowAtDisp	REAL				False	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT				False	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL				False	HMI: show slow-down display
UwSlow_at_Metre	REAL				False	Unwinder: slow-down starts at (m)
UwSlowReq	BOOL				False	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL				False	Rewinder: slow-down starts at diameter (mm)
RwStopAtDiam	REAL				False	Rewinder: stop command at diameter (mm)
RwMaxDiam	REAL				False	Rewinder max diameter protection (mm)
RwMaxStop	BOOL				False	Rewinder max diameter STOP (latched)
RwMaxSlow	BOOL				False	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL				False	
RwThickMin_mm	REAL				False	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL				False	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL				False	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL				False	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT				False	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT				False	Learning: failed stops before alarm
GSM_Learned_OK	BOOL				False	Rewinder learning fault (alarm 16)
RwThick_mm	REAL				False	Learned paper thickness (mm)

1-7-1-1-2.General

0 Remote 10.146.233.11 ... 1s clock.



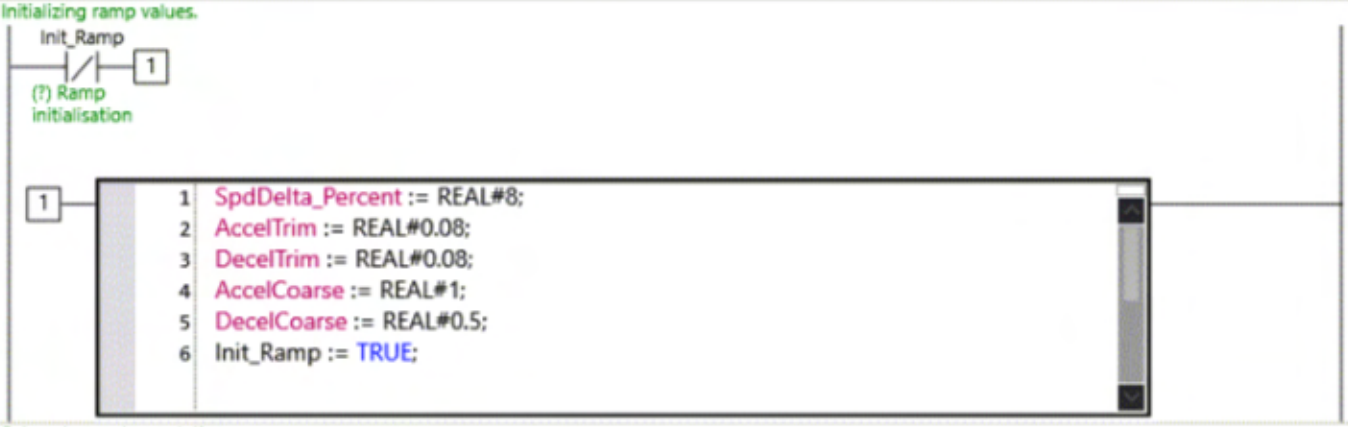
1 100ms clock.



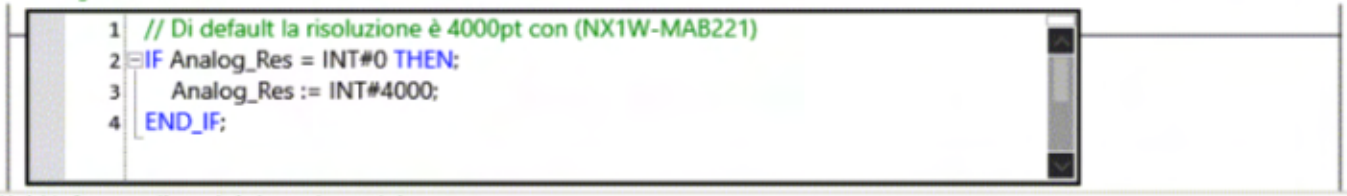
2 10ms clock.



3 Initializing ramp values.

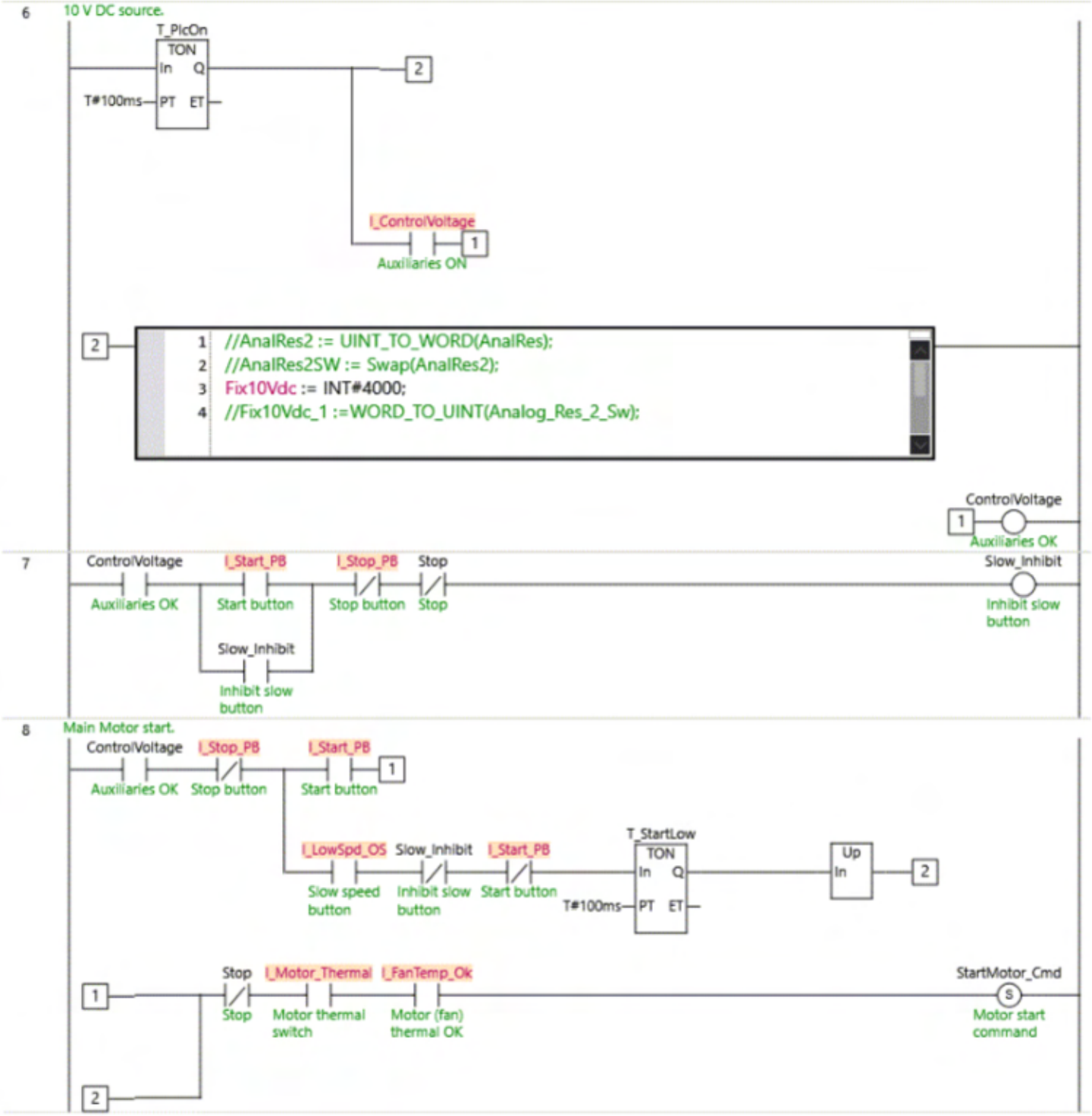


4 Set analog scale resolution.



5 Variable for converting m/min to mm/sec and vice versa.




6
10 V DC source.
T_PlcOn
TON
In Q
T#100ms PT ET
2
I_ControlVoltage
1
Auxiliaries ON
2
1 //AnalRes2 := UINT_TO_WORD(AnalRes);
2 //AnalRes2SW := Swap(AnalRes2);
3 Fix10Vdc := INT#4000;
4 //Fix10Vdc_1 :=WORD_TO_UINT(Analog_Res_2_Sw);
ControlVoltage
1
Auxiliaries OK
7
ControlVoltage
Auxiliaries OK
I_Start_PB
Start button
I_Stop_PB
Stop button
Stop
Stop
Slow_Inhibit
Inhibit slow button
Slow_Inhibit
Inhibit slow button
8
Main Motor start.
ControlVoltage
Auxiliaries OK
I_Stop_PB
Stop button
I_Start_PB
Start button
1
I_LowSpd_OS
Slow speed button
Slow_Inhibit
Inhibit slow button
I_Start_PB
Start button
T_StartLow
TON
In Q
T#100ms PT ET
Up
In
2
1
Stop
Stop
I_Motor_Thermal
Motor thermal switch
I_FanTemp_Ok
Motor (fan) thermal OK
StartMotor_Cmd
S
Motor start command
2

9

```
// v6m: STOP button = soft stop. StopNow replaces I_Stop_PB in rung 9.
StopNow := FALSE;
T_StopLong(In := I_Stop_PB, PT := T#1s);

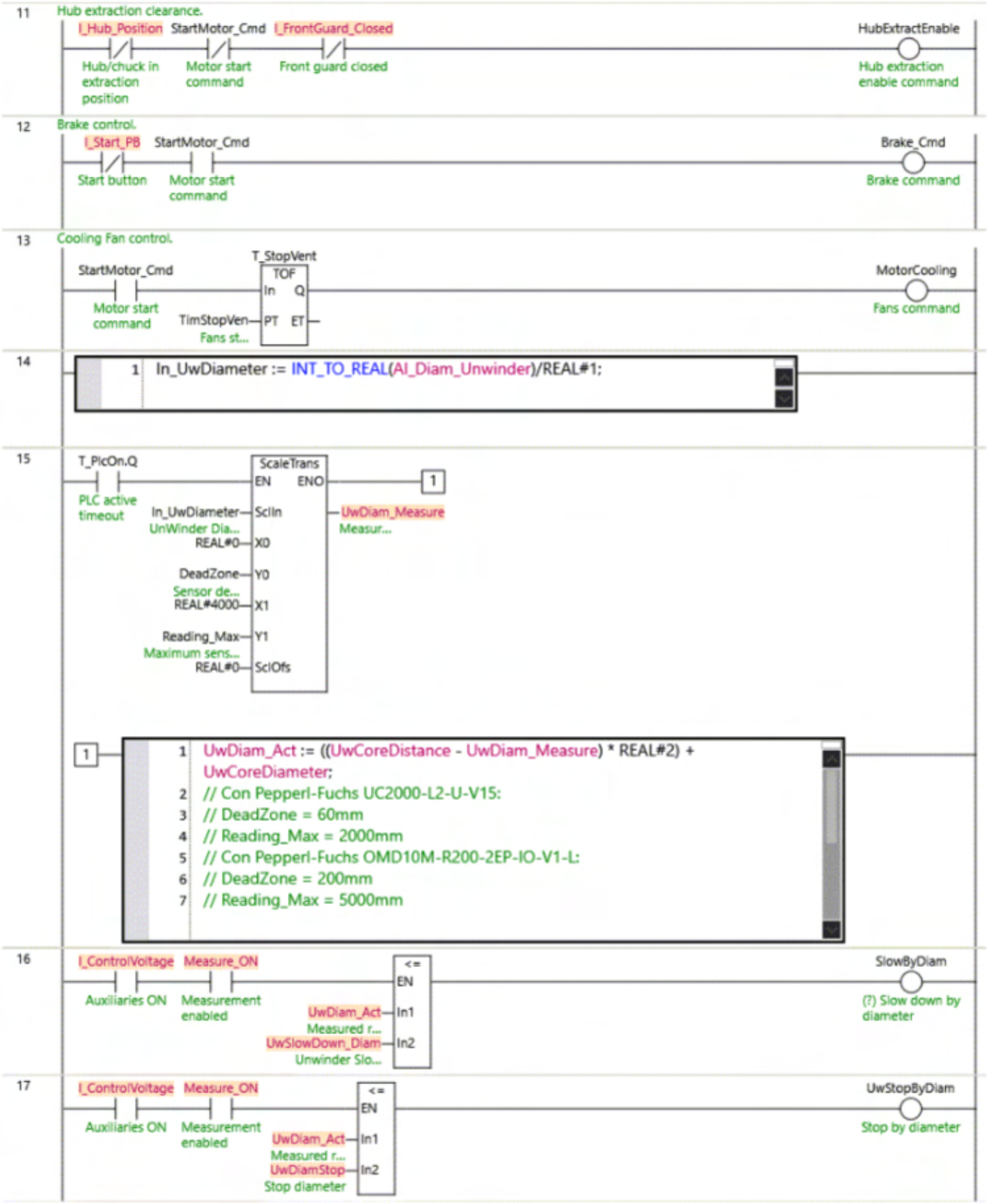
IF NOT StartMotor_Cmd THEN
    SoftStop := FALSE;
ELSIF I_Stop_PB AND NOT StopPB_Mem THEN          // STOP pressed (edge)
    IF SoftStop OR NOT SoftStopEnable OR I_LowSpd_OS
      OR (Velocity_M_min <= SoftStopEndSpd) THEN
        StopNow := TRUE;                         // 2nd press / jog / slow / disabled
    ELSE
        SoftStop := TRUE;                        // start soft stop
    END_IF;
END_IF;
StopPB_Mem := I_Stop_PB;

IF T_StopLong.Q THEN
    StopNow := TRUE;                             // held 1 s = immediate
END_IF;
IF SoftStop AND I_Start_PB THEN
    SoftStop := FALSE;                           // START cancels the soft stop
END_IF;

T_SoftStop(In := SoftStop, PT := T#120s);
IF SoftStop AND ((Velocity_M_min <= SoftStopEndSpd) OR T_SoftStop.Q) THEN
    StopNow := TRUE;                             // almost stopped (or timeout)
END_IF;
```

10 Main Motor stop.




11
Hub extraction clearance.
I_Hub_Position
Hub/chuck in extraction position
StartMotor_Cmd
Motor start command
I_FrontGuard_Closed
Front guard closed
HubExtractEnable
Hub extraction enable command
12
Brake control.
I_Start_PB
Start button
StartMotor_Cmd
Motor start command
Brake_Cmd
Brake command
13
Cooling Fan control.
StartMotor_Cmd
Motor start command
T_StopVent
TOF
In
Q
TimStopVen
Fans st...
PT
ET
MotorCooling
Fans command
14
1 In_UwDiameter := INT_TO_REAL(AI_Diam_Unwinder)/REAL#1;
15
T_PlcOn.Q
PLC active timeout
ScaleTrans
EN
ENO
1
In_UwDiameter
UnWinder Dia...
ScIn
UwDiam_Measure
Measur...
REAL#0
X0
DeadZone
Y0
Sensor de...
REAL#4000
X1
Reading_Max
Maximum sens...
Y1
REAL#0
ScOfs
1
1 UwDiam_Act := ((UwCoreDistance - UwDiam_Measure) * REAL#2) +
UwCoreDiameter;
2 // Con Pepperl-Fuchs UC2000-L2-U-V15:
3 // DeadZone = 60mm
4 // Reading_Max = 2000mm
5 // Con Pepperl-Fuchs OMD10M-R200-2EP-IO-V1-L:
6 // DeadZone = 200mm
7 // Reading_Max = 5000mm
16
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwSlowDown_Diam
Unwinder Slo...
In2
SlowByDiam
(?) Slow down by diameter
17
I_ControlVoltage
Auxiliaries ON
Measure_ON
Measurement enabled
<=
EN
UwDiam_Act
Measured r...
In1
UwDiamStop
Stop diameter
In2
UwStopByDiam
Stop by diameter

1-7-1-1-3.Motor_Manager

0 Enable encoder reading.



1 Calculation of mm per encoder pulse.

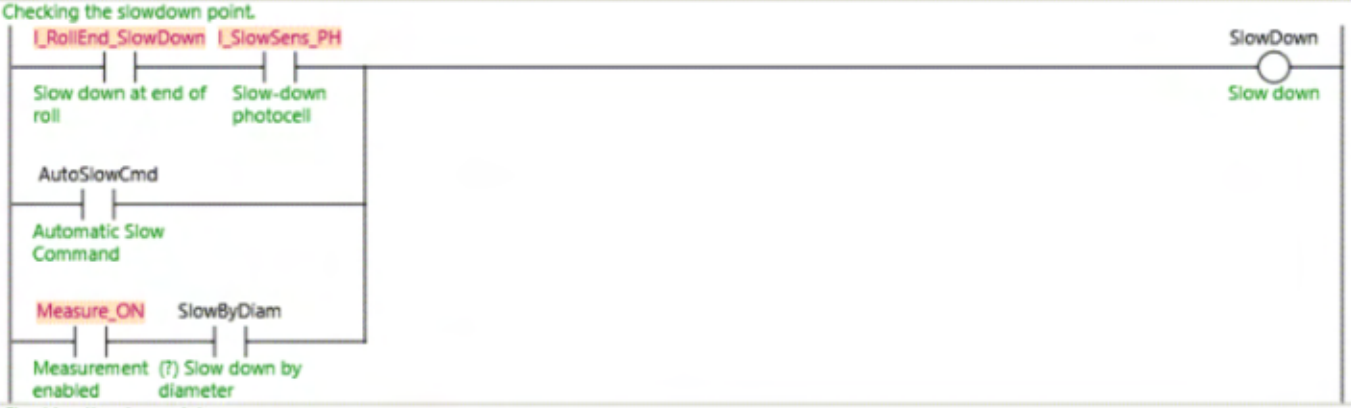
```
IF Encoder_Pulse = INT#0 THEN;
    Encoder_Pulse :=INT#1;
END_IF;
// Encoder Standard 400mm
IF  EncoderWheel_Diam = REAL#0 THEN;
    EncoderWheel_Diam := REAL#400;
END_IF;

EncoderPulse_to_mm := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) *
REAL#4);
```

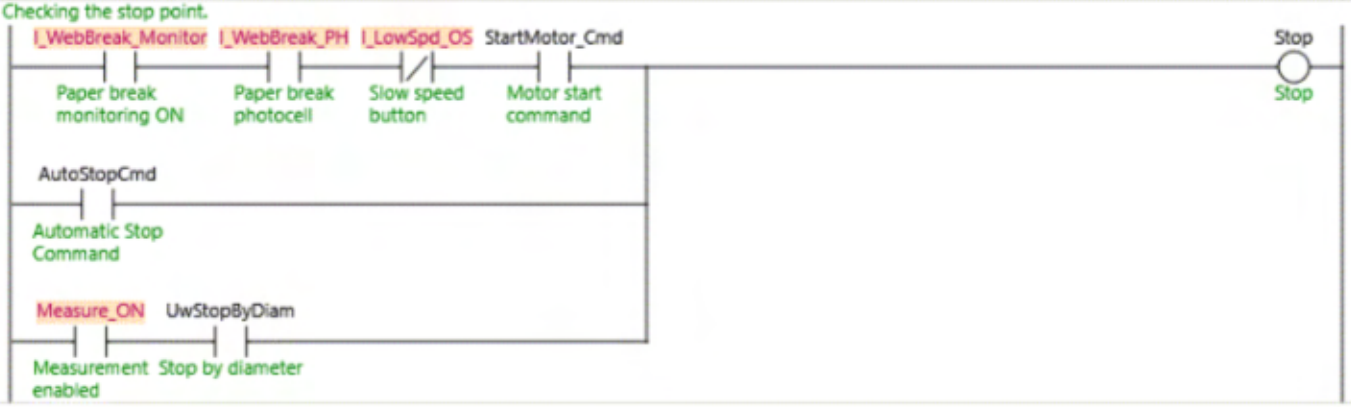
2 Reset encoder counter.



3 Checking the slowdown point.



4 Checking the stop point.



5 Check the maximum and minimum values for working speed and slow speed.

```
IF VelMax < REAL#600 THEN;
      VelMax := REAL#600;
   ELSIF VelMax > REAL#1000 THEN;
      VelMax := REAL#1000;
END_IF;
Vel_Max_mms := VelMax * REAL#1000 / REAL#60;

IF JogSpd_SP < REAL#10 THEN;
      JogSpd_SP := REAL#10;
ELSIF JogSpd_SP > REAL#70 THEN;
   JogSpd_SP := REAL#70;
END_IF;

Output_Ratio := (VelMax * Mt_To_mm) / UINT_TO_REAL(Analog_Res): //REAL#4000;
Input_Ratio := UINT_TO_REAL(Analog_Res) / VelMax ;
```

6 Reading the current value of the potentiometers. v6n: requested pot ranges, limited, applied only with the motor stopped !

```
IF WorkSpd_Max_Set < 50.0 THEN
   WorkSpd_Max_Set := 50.0;
ELSIF WorkSpd_Max_Set > VelMax THEN
   WorkSpd_Max_Set := VelMax;
END_IF;

IF FinalSpd_Max_Set < 5.0 THEN
   FinalSpd_Max_Set := 5.0;
ELSIF FinalSpd_Max_Set > WorkSpd_Max_Set THEN
   FinalSpd_Max_Set := WorkSpd_Max_Set;
END_IF;

IF NOT StartMotor_Cmd THEN
   WorkSpd_Max := WorkSpd_Max_Set;          // motor stopped: apply
   FinalSpd_Max := FinalSpd_Max_Set;
END_IF;

SpdMax_Pending := (WorkSpd_Max <> WorkSpd_Max_Set)
            OR (FinalSpd_Max <> FinalSpd_Max_Set);

// v6i: pots scaled to their own full scale (m/min)

IF Pot1_Work_Velocity > INT#5 THEN
   Pot1_Mem := REAL_TO_INT(INT_TO_REAL(Pot1_Work_Velocity) * WorkSpd_Max /
VelMax);
ELSE
   Pot1_Mem := INT#0;
END_IF;

IF Pot2_Slow_Velocity > INT#5 THEN
   Pot2_Mem := REAL_TO_INT(INT_TO_REAL(Pot2_Slow_Velocity) * FinalSpd_Max /
VelMax);
ELSE
   Pot2_Mem := INT#0;
END_IF;
```

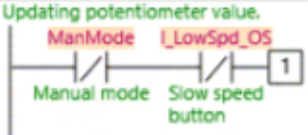
7 Updating the set speed.

```
IF ManMode = FALSE THEN;

    Speed_Display := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
    LowSpd_Display := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
ELSE;
   Speed_Display := Pot1_Work_Velocity / (Analog_Res / UINT#40); // v6i: raw pot. %
   LowSpd_Display := Pot2_Slow_Velocity / (Analog_Res / UINT#40);

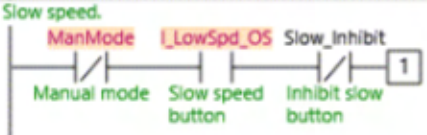
END_IF;
```

8 Updating potentiometer value.




```
IF SoftStop THEN
    VelSet := 0.0;                    // v6m: soft stop
ELSIF SlowDown = FALSE THEN
    VelSet := UINT_TO_REAL(Pot1_Mem) / Input_Ratio;
ELSE
    VelSet := UINT_TO_REAL(Pot2_Mem) / Input_Ratio;
END_IF;
VelSet_mms := VelSet * Mt_To_mm;
```

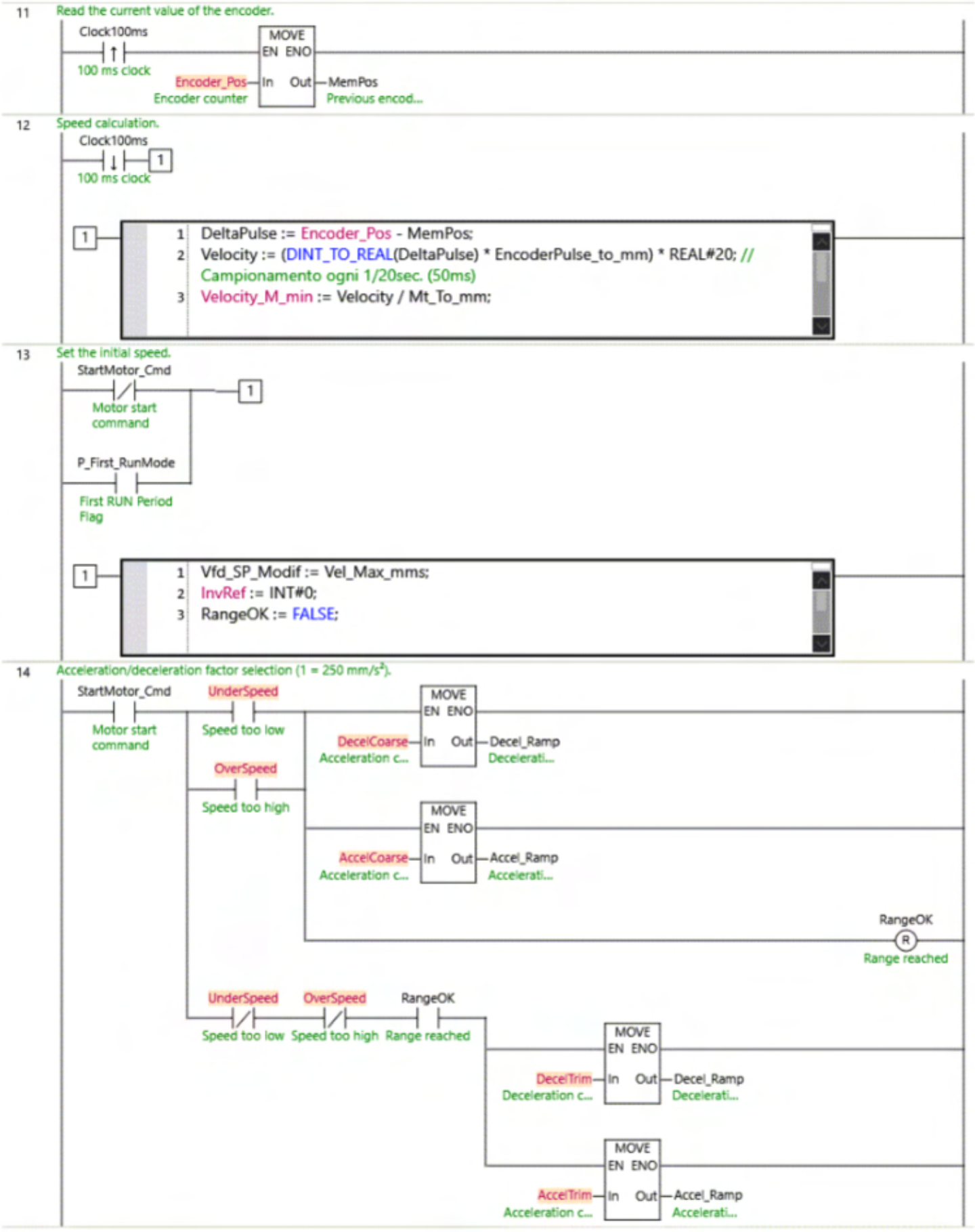
9 Slow speed.




```
VelSet := JogSpd_SP;
VelSet_mms := VelSet * Mt_To_mm;
```

10 Check the difference between the set speed and the detected speed.

```
//SpdDelta := VelSet_mms * SpdDelta_Percent / REAL#100;
SpdDelta := SpdDelta_Percent;
SpdDeltaMax := VelSet + SpdDelta;
SpdDeltaMin := VelSet - SpdDelta;
// Too fast
IF Velocity_M_min > SpdDeltaMax THEN;
    OverSpeed := TRUE;
ELSE
    OverSpeed := FALSE;
END_IF;
// Too slow
IF Velocity_M_min < SpdDeltaMin THEN;
    UnderSpeed := TRUE;
ELSE
    UnderSpeed := FALSE;
END_IF;
```


11
Read the current value of the encoder.
Clock100ms
100 ms clock
MOVE
EN ENO
Encoder_Pos
Encoder counter
In Out
MemPos
Previous encod...
12
Speed calculation.
Clock100ms
100 ms clock
1
DeltaPulse := Encoder_Pos - MemPos;
Velocity := (DINT_TO_REAL(DeltaPulse) * EncoderPulse_to_mm) * REAL#20; //
Campionamento ogni 1/20sec. (50ms)
Velocity_M_min := Velocity / Mt_To_mm;
13
Set the initial speed.
StartMotor_Cmd
Motor start command
P_First_RunMode
First RUN Period Flag
Vfd_SP_Modif := Vel_Max_mms;
InvRef := INT#0;
RangeOK := FALSE;
14
Acceleration/deceleration factor selection (1 = 250 mm/s²).
StartMotor_Cmd
Motor start command
UnderSpeed
Speed too low
OverSpeed
Speed too high
DecelCoarse
Acceleration c...
Decel_Ramp
Decelerati...
AccelCoarse
Accel_Ramp
Accelerati...
RangeOK
Range reached
DecelTrim
Deceleration c...
AccelTrim

15 Optional faster decel

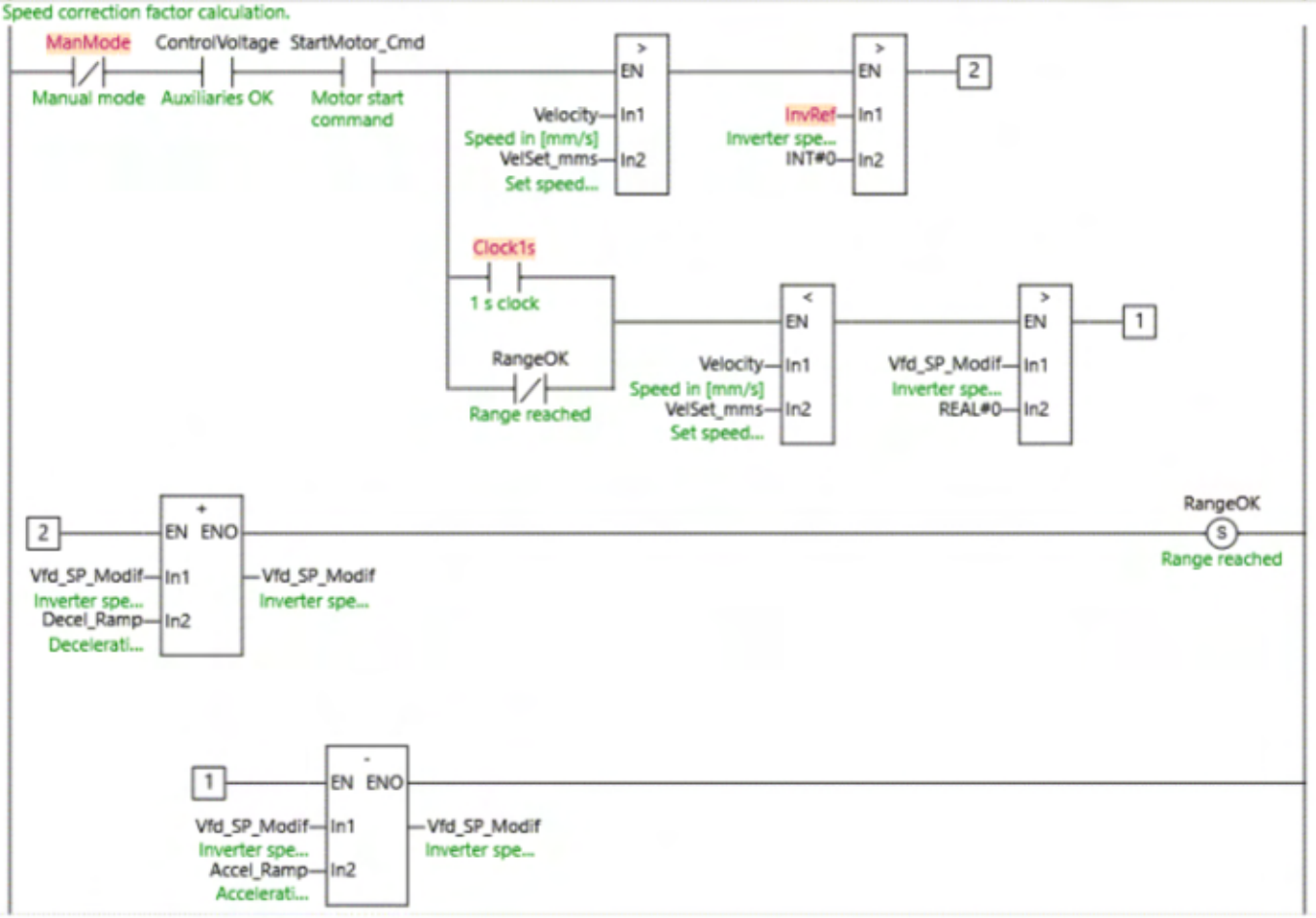
```
// v6m: soft stop decel (0 = keep DecelCoarse / DecelTrim)
IF SoftStop AND (SoftStopDecel > 0.0) THEN
    Decel_Ramp := SoftStopDecel / 250.0;          // 1 = 250 mm/s2 (4 ms task)
END_IF;
```

16 v6o: Decel ramp limited to the learned drive decel

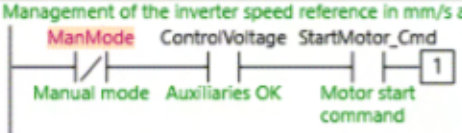
```
// v6o: never ask more decel than the drive can give (anti wind-up).
// RwSlowDecelLrn = RwSlowDecelK x rewinder diameter (mm/s2), AutoStop section 5c.
// 1.25 margin: the measured decel can still grow, so the learning never only ratchets down.

IF RwSlowDecelLrn > 0.0 THEN
    IF Decel_Ramp > RwSlowDecelLrn * 1.25 / 250.0 THEN
        Decel_Ramp := RwSlowDecelLrn * 1.25 / 250.0;   // 1 = 250 mm/s2 (4 ms task)
    END_IF;
END_IF;
```

17 Speed correction factor calculation.

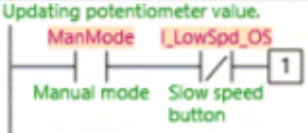


18 Management of the inverter speed reference in mm/s and conversion to analog points (0–4000 points = 0–10 VDC = 0–10,000 mm/s).




```
1  //InvRef2 := REAL_TO_UINT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
2  //InvRefWW := UINT_TO_WORD(InvRef2);
3  //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
4  //InvRef := WORD_TO_UINT(InvRefSW);
5  InvRef := REAL_TO_INT((Vel_Max_mms - Vfd_SP_Modif ) / Output_Ratio);
```

19 Updating potentiometer value.



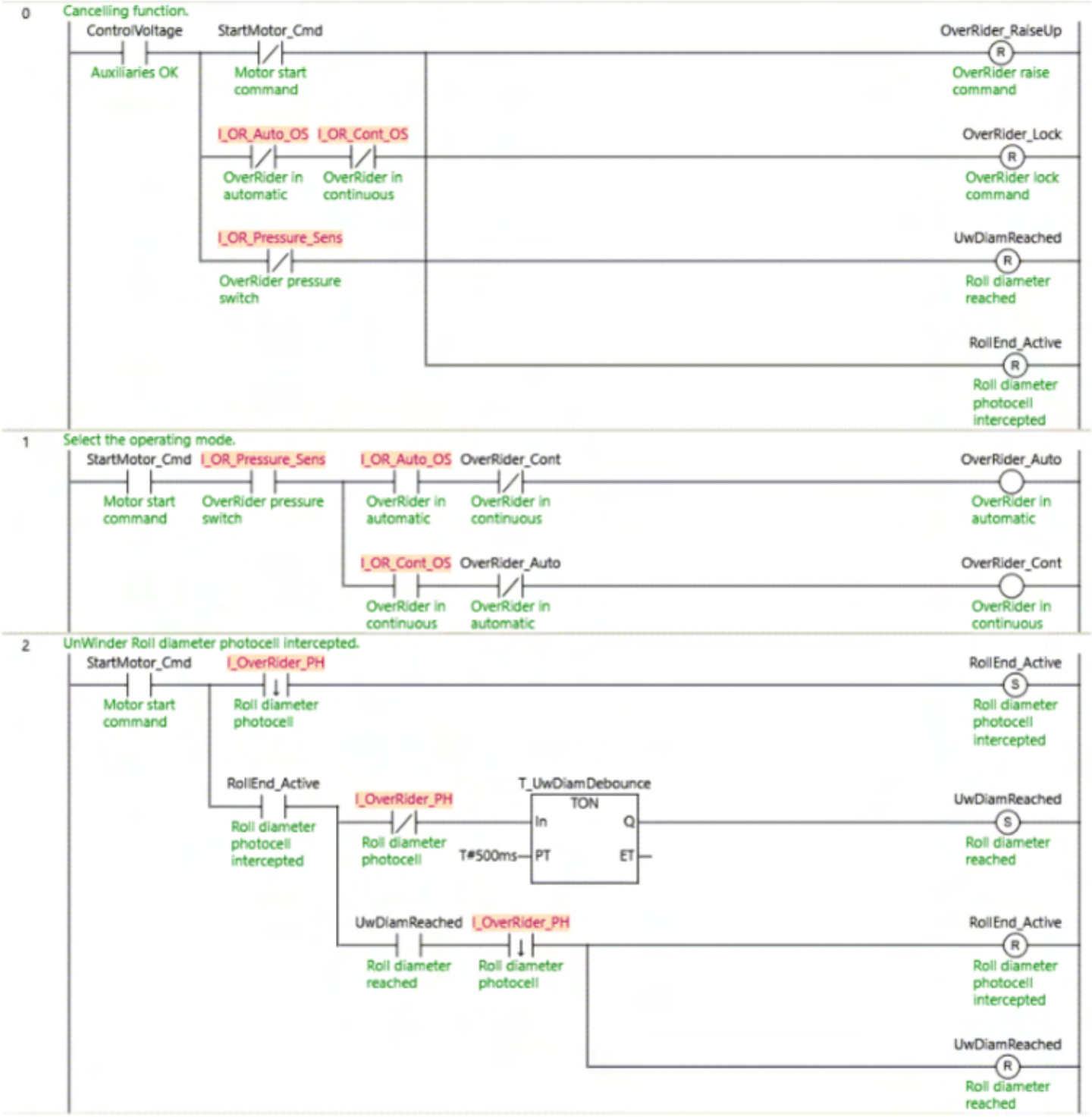

```
1  IF SoftStop THEN
2      InvRef := INT#0;                    // v6m: drive ramps down on its own
3  ELSIF SlowDown = FALSE THEN
4      InvRef := Pot1_Work_Velocity;
5  ELSE
6      InvRef := Pot2_Slow_Velocity;
7  END_IF;
```

20 Slow speed.

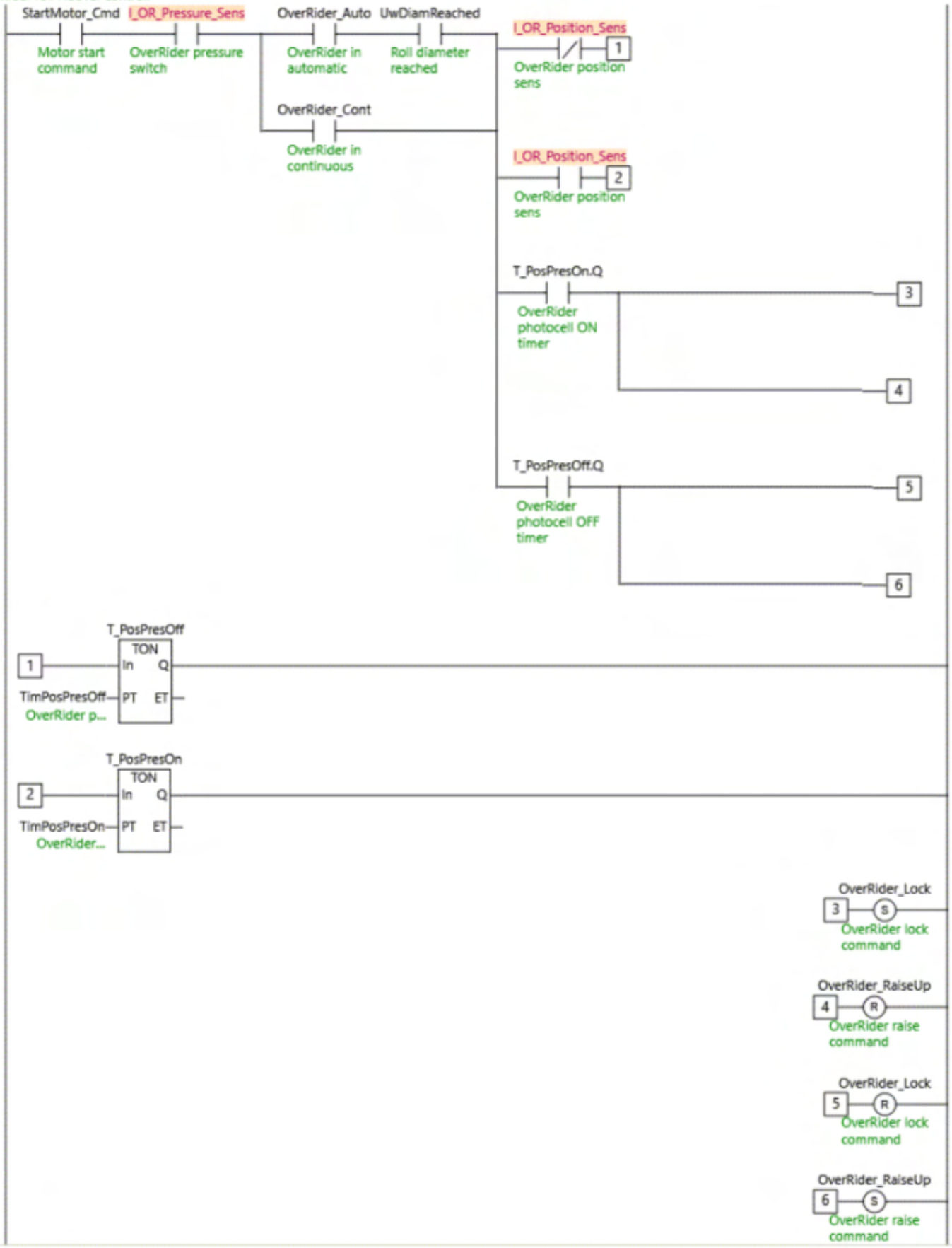



```
1  VelSet := JogSpd_SP;
2  VelSet_mms := VelSet * Mt_To_mm;
3  //InvRef2 := REAL_TO_UINT((VelSet_mms) / Output_Ratio);
4  //InvRefWW := UINT_TO_WORD(InvRef2);
5  //InvRefSW :=Swap(UINT_TO_WORD(InvRef2));
6  //InvRef := WORD_TO_UINT(InvRefSW);
7
8  InvRef := REAL_TO_INT(VelSet_mms / Output_Ratio);
```

1-7-1-1-4.OverRider_Manager

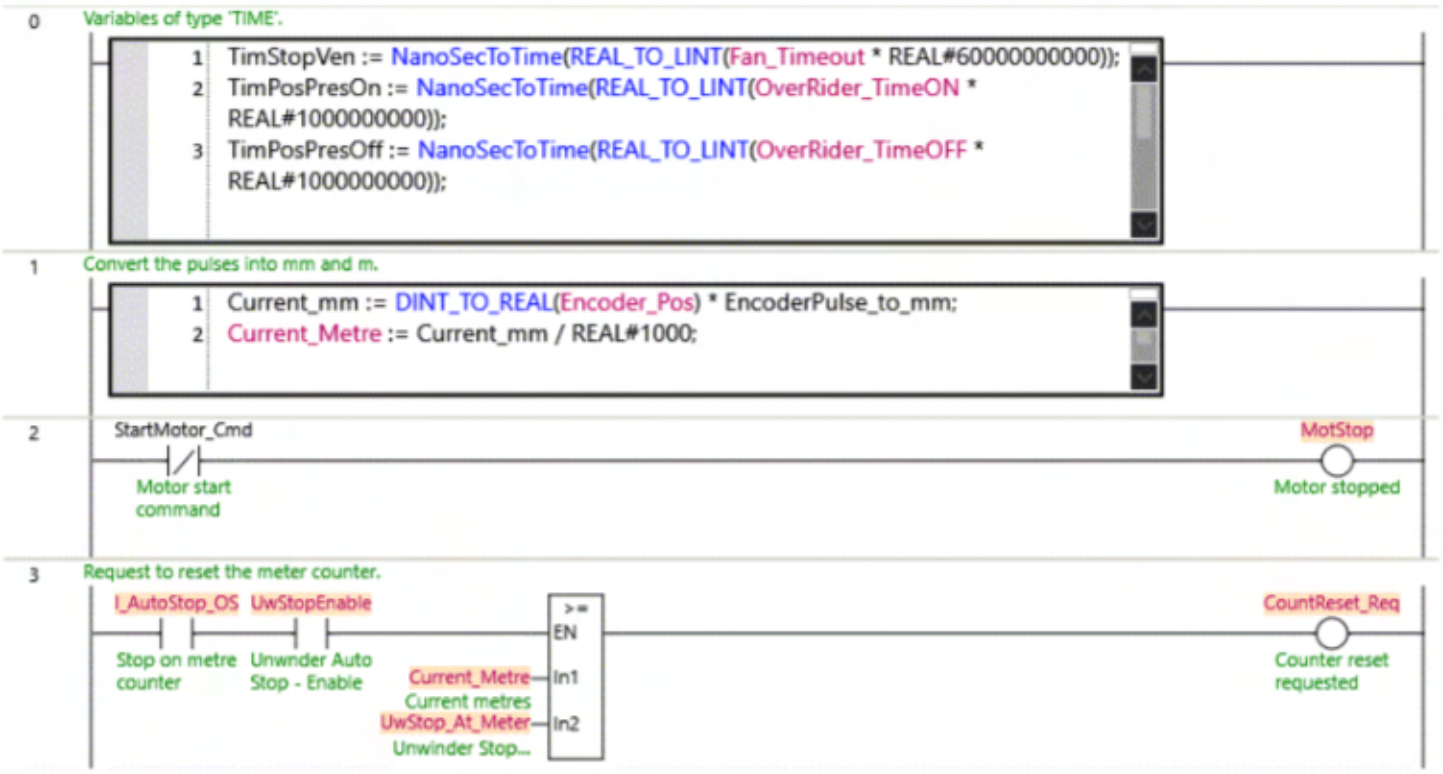


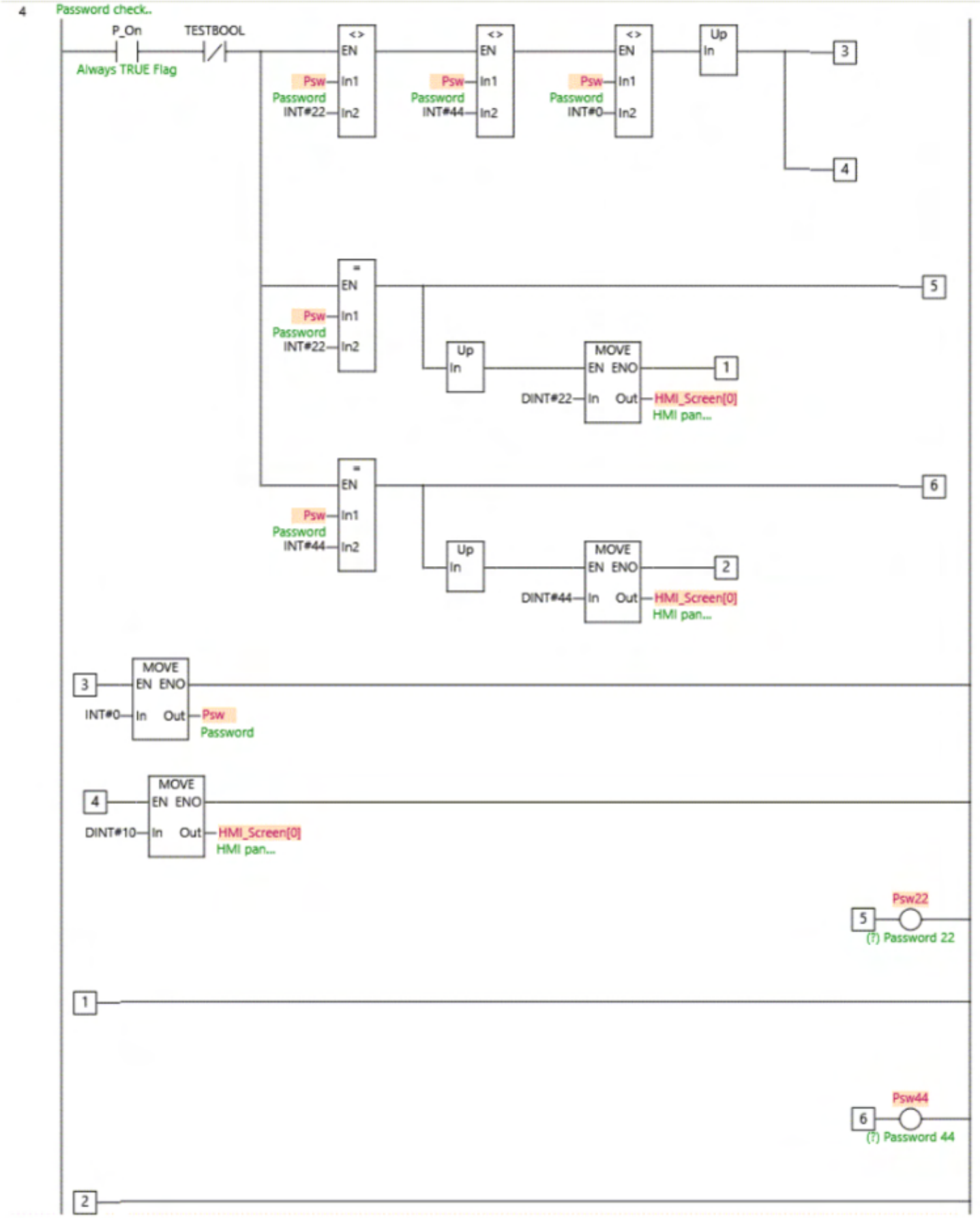
3 Rider roll Hoover control.

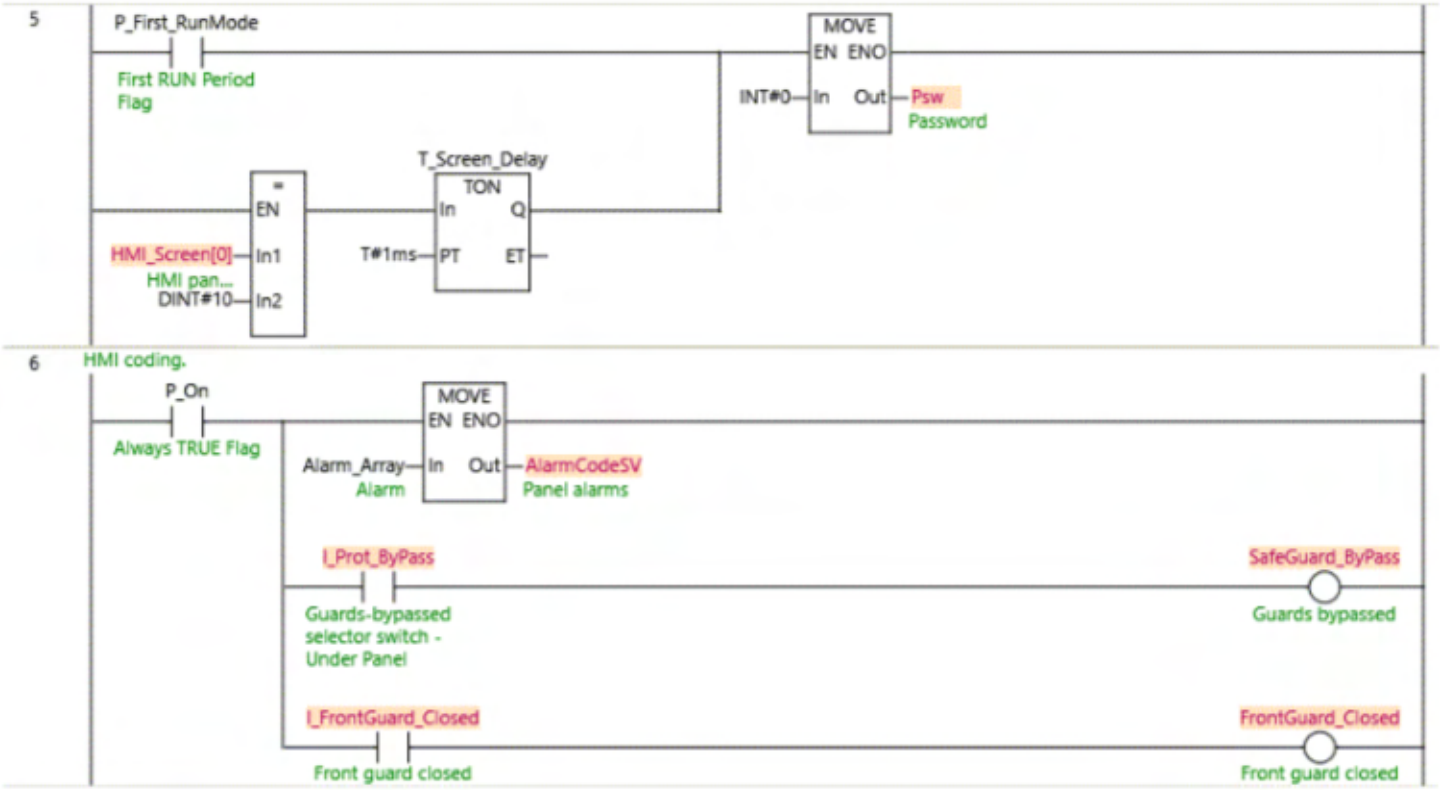



4
Rider roll raise command.
ControlVoltage
Auxiliaries OK
OverRider_RaiseUp
S
OverRider raise command
5
I_ControlVoltage
Auxiliaries ON
OverRider_RaiseUp
R
OverRider raise command

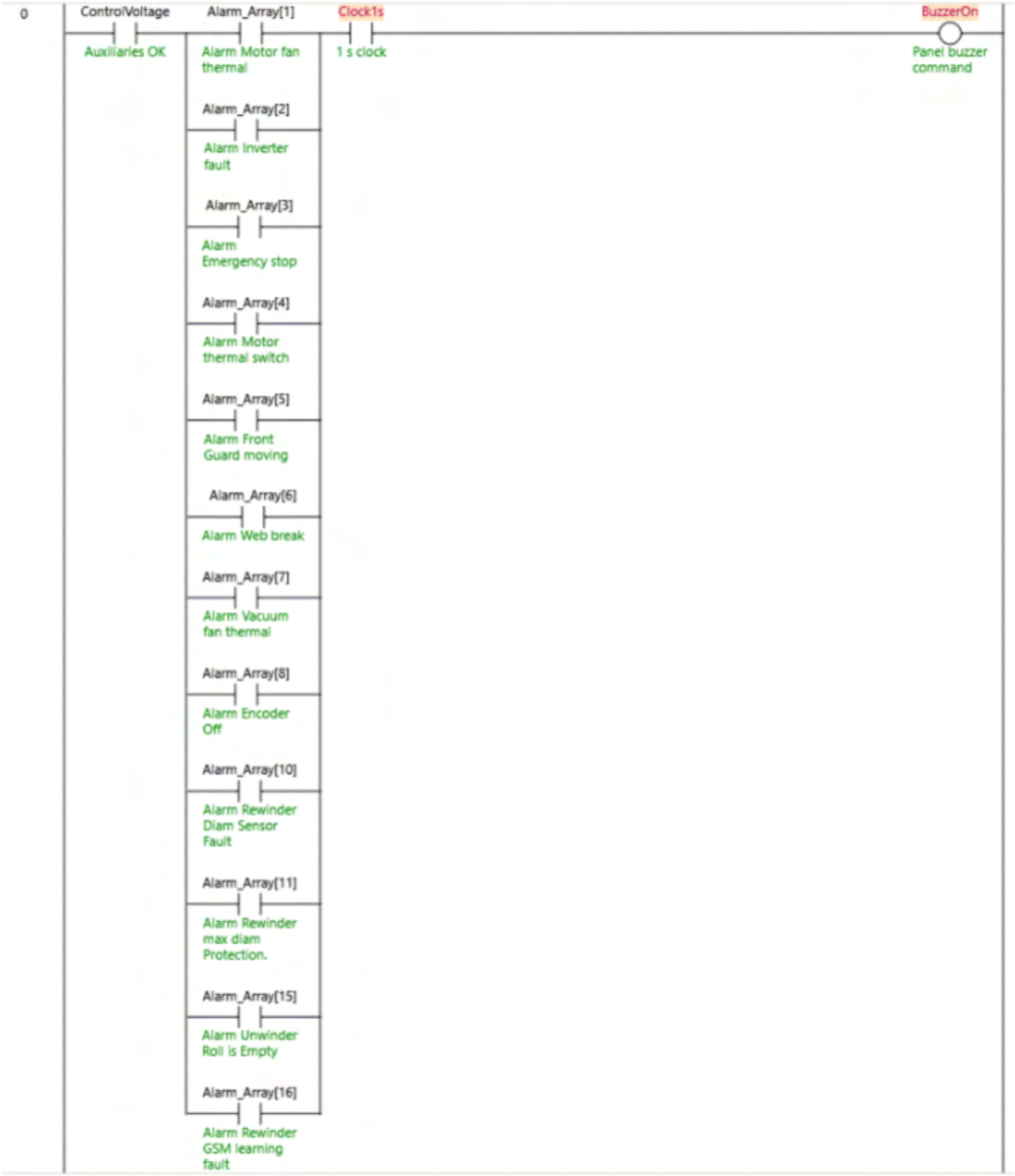
1-7-1-1-5.Console



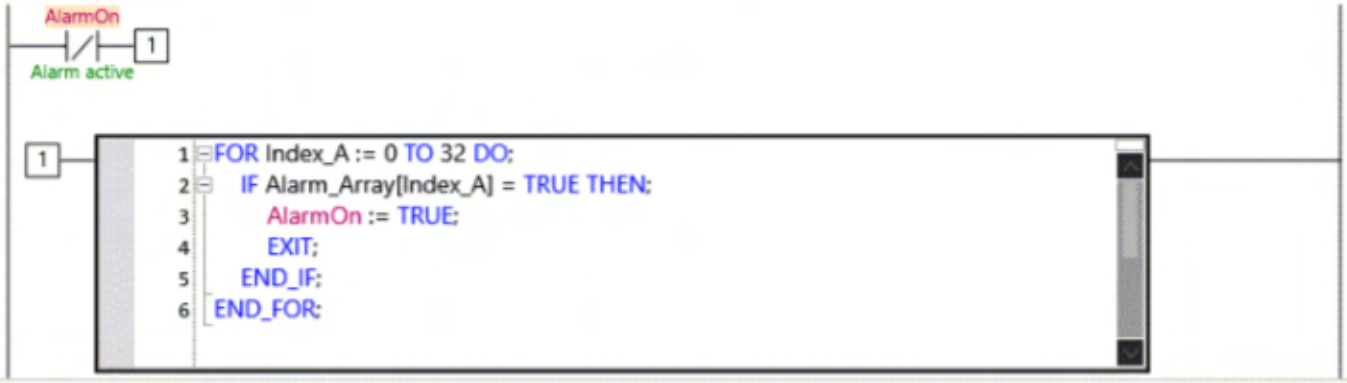

4
Password check..
P_On
TESTBOOL
Always TRUE Flag
<>
EN
Psw
Password
In1
In2
INT#22
INT#44
INT#0
Up
In
3
4
=
5
6
MOVE
EN ENO
In Out
1
2
DINT#22
DINT#44
HMI_Screen[0]
HMI pan...
INT#0
Psw
Password
DINT#10
Psw22
(?) Password 22
Psw44
(?) Password 44


5
P_First_RunMode
First RUN Period Flag
MOVE
EN ENO
INT#0
In
Out
Psw
Password
T_Screen_Delay
=
EN
In1
In2
HMI_Screen[0]
HMI pan...
DINT#10
TON
In
Q
T#1ms
PT
ET
6
HMI coding.
P_On
Always TRUE Flag
MOVE
EN ENO
Alarm_Array
Alarm
In
Out
AlarmCodeSV
Panel alarms
I_Prot_ByPass
Guards-bypassed selector switch - Under Panel
SafeGuard_ByPass
Guards bypassed
I_FrontGuard_Closed
Front guard closed
FrontGuard_Closed
Front guard closed

1-7-1-1-6.Alarms



1 Check Present alarms.



2 Alarm has been called off.



3 Motor cooling Fan Alarm.



4 Inverter fault.



5 Machine in emergency mode.



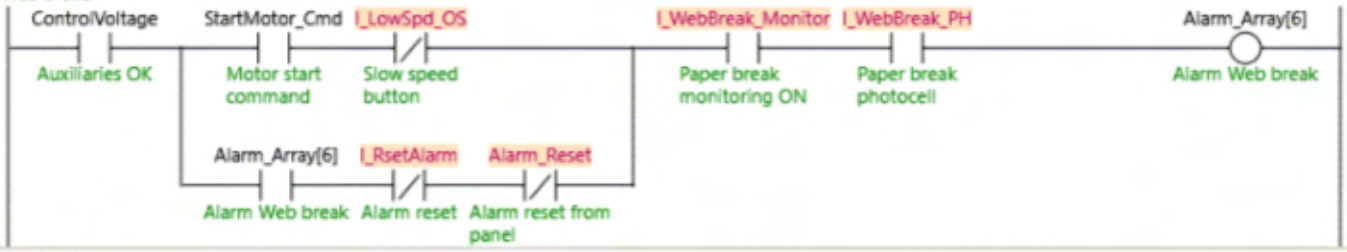
6 Motor thermal cutout.



7



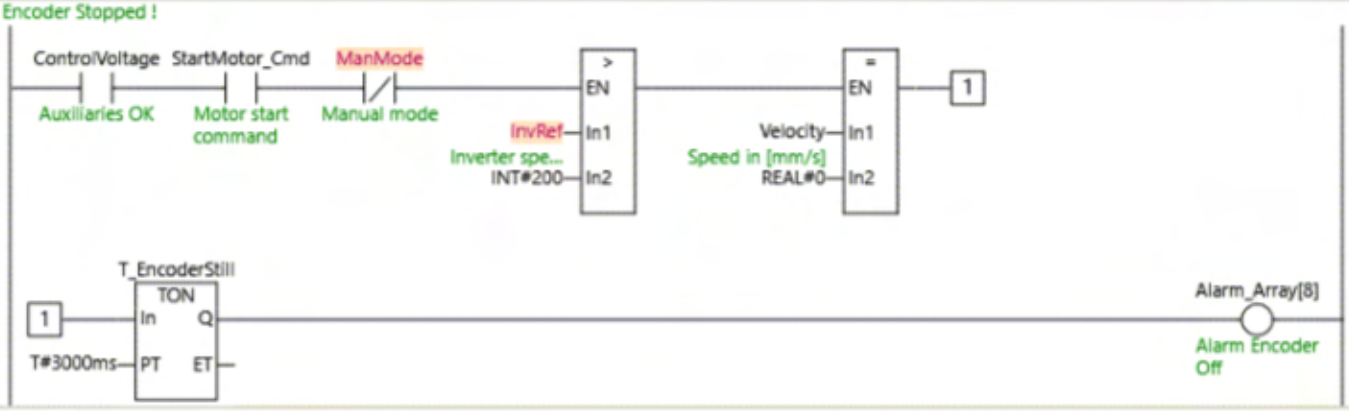
8 Web Brake.



9 Cooling fan.



10 Encoder Stopped !



11 Open-ended roll holder joint.



12 Rewinder Diameter Sensor Fault.



13 Rewinder Diameter Sensor Fault - Reset



14 Rewinder max diameter Machine protection.



15 Rewinder max diameter Machine protection - Reset



16 Rewinder GSM learning fault



17 Rewinder learning fault - Reset



18 Hub not in position.



19 Emergency Stop by Holding Stop PB 2s



20 Slow-down photocell active.



21 End-of-roll Alarm.



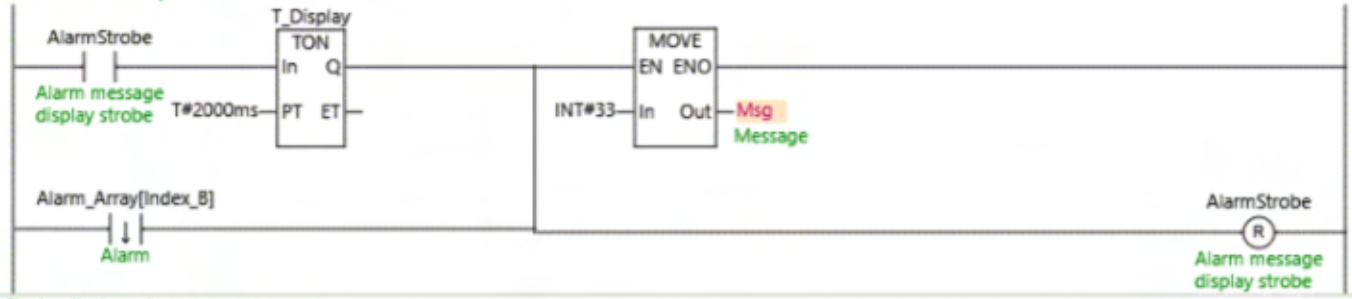
22 Unwinder Roll is Empty



23 Unwinder Roll is Empty - Reset



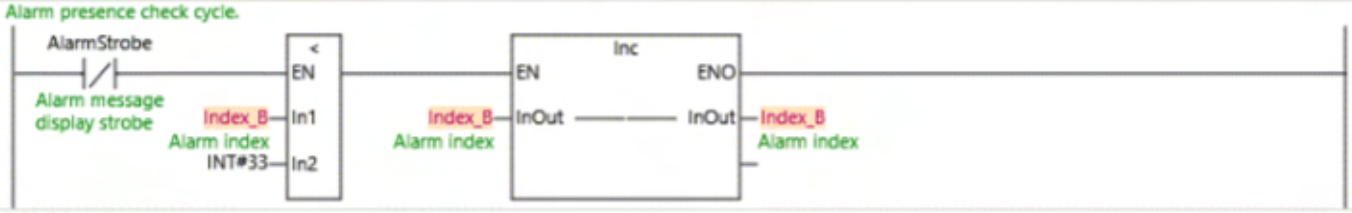
24 Active end-of-roll photocell.



25 Strobe displays alarm message.



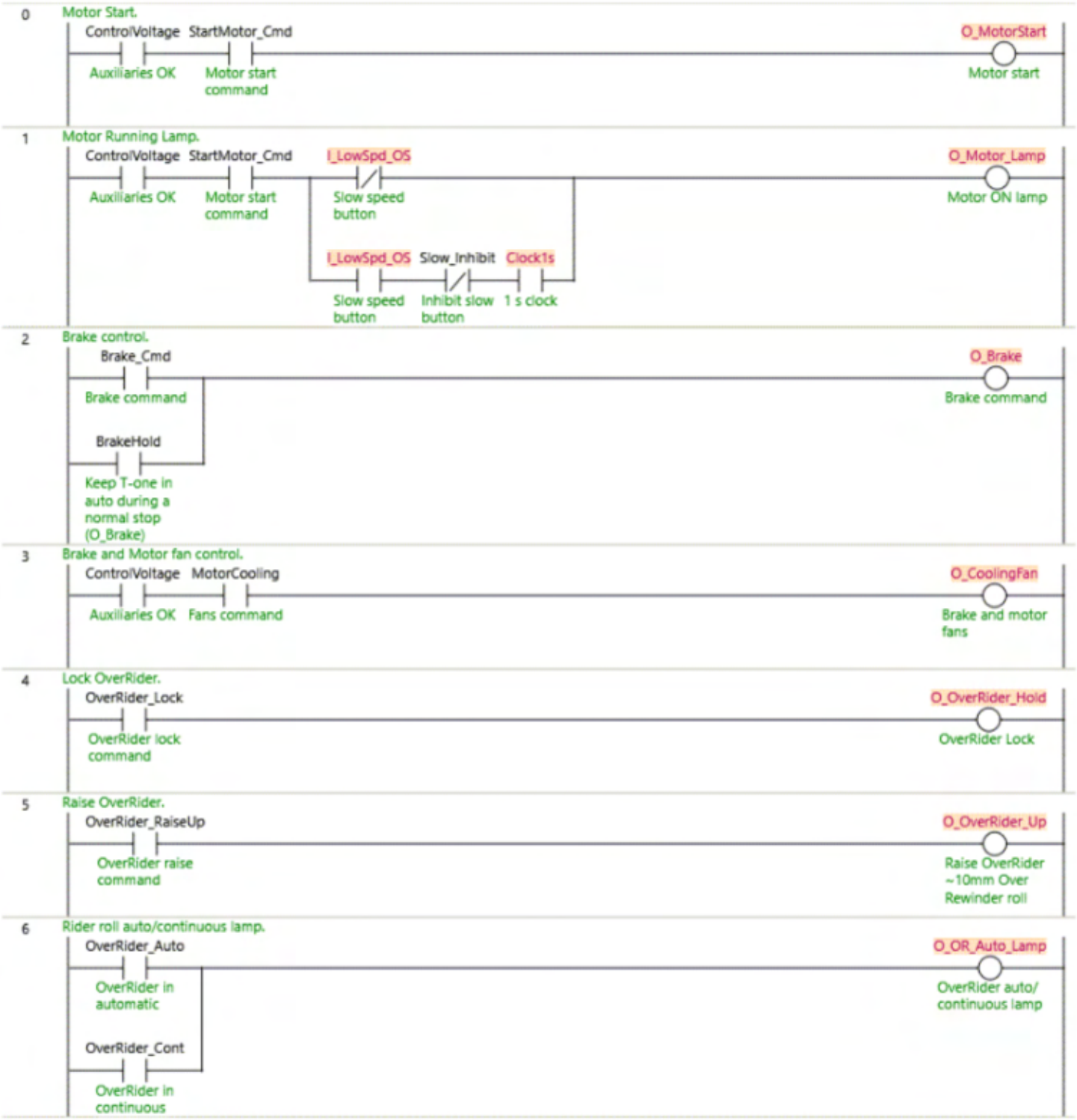
26 Alarm presence check cycle.



27 Initializing alarm message.



1-7-1-1-7.Output_Manager

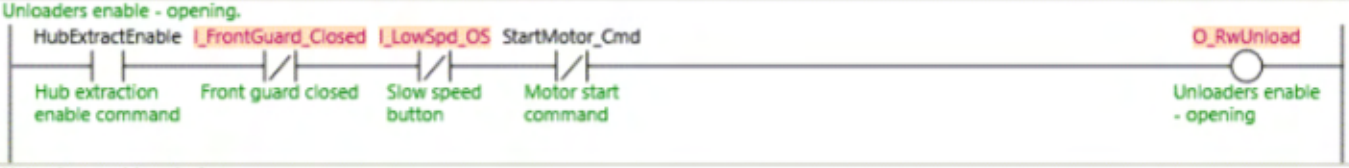



7
Alarm lamp.
ControlVoltage
Auxiliaries OK
O_Alarm_Array
Alarm lamp
Alarm_Array[1]
Alarm Motor fan thermal
Clock1s
1 s clock
Alarm_Array[2]
Alarm Inverter fault
Alarm_Array[3]
Alarm Emergency stop
Alarm_Array[4]
Alarm Motor thermal switch
Alarm_Array[5]
Alarm Front Guard moving
Alarm_Array[6]
Alarm Web break
Alarm_Array[7]
Alarm Vacuum fan thermal
Alarm_Array[8]
Alarm Encoder Off
Alarm_Array[10]
Alarm Rewinder Diam Sensor Fault
Alarm_Array[11]
Alarm Rewinder max diam Protection.
Alarm_Array[15]
Alarm Unwinder Roll is Empty
Alarm_Array[16]
Alarm Rewinder GSM learning fault

8 Guards bypassed lamp.



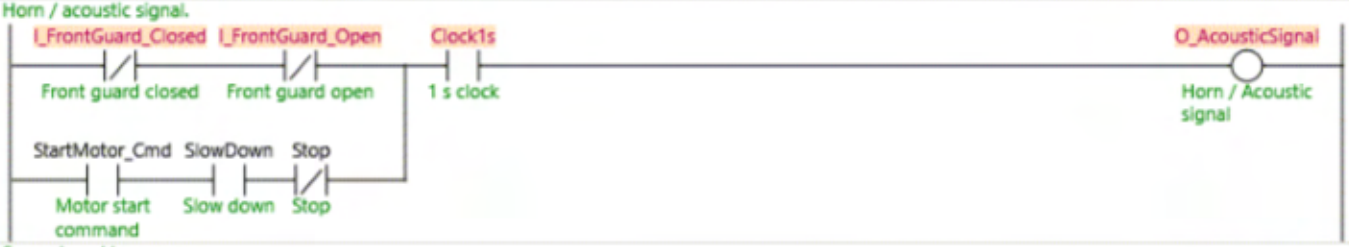
9 Unloaders enable - opening.



10 Unloaders enable - closing.



11 Horn / acoustic signal.



12 Second acc/dec ramp.



13 Front guard bypass.



1-7-1-1-8.AutoStop_Control

0

```
(*
=============================================================
====================
  AutoStop_Control  v6l  -  Automatic slow-down / stop
  Project : 04820KA001 Mono Canada II  (NX1P2, PrimaryTask 4 ms)
  Rewinder (Rw) : SICK DT35 0-10 V -> AI_Diam_Rewinder (AD2603 Ch2)
  Unwinder (Uw) : meter counter mode + sonar diameter (UwDiam_Act)

  NAMING  Uw_ = unwinder   Rw_ = rewinder
        _mm = millimeters   _m = meters
        _Pls = encoder pulses (counts)   _Pps = pulses per second
        _Pps2 = pulses per second squared

  v6l: previous value saved before every TEACH CORE (RwAxisDist_Prev,
       UwCoreDist_Prev, Retain) + UNDO buttons (RwTeachUndo,
       UwTeachUndo): swap current <-> previous. State 4 = RESTORED.
  v6k: unwinder TEACH CORE (Core_Measure) works like the rewinder one:
       same checks, 2 s stable with sonar AVERAGE, status UwTeachState.
       Replaces General rung 15 (one immediate sonar reading).
  v6j: TEACH CORE (section 14): HMI button RwTeachCore, empty core on the
       rewinder, machine stopped -> after 2 s stable:
       RwAxisDist := RwDist + RwCoreOD / 2. Status in RwTeachState.
  v6i: auto: pots scaled to WorkSpd_Max / FinalSpd_Max (Motor_Manager rung 6,
       Pot1_Mem / Pot2_Mem). ManMode keeps the raw pots (percent),
       so section 6 uses the raw pots in ManMode.
  v6h: BrakeHold keeps O_Brake (T-one "reel brake automatic") ON during a
       normal stop until the web is almost stopped (< 5 m/min, max 15 s).
       Before: T-one switched to "stop" with the run command, while the
       rewinder was decelerating hard -> unwinder overran, web on the
       floor (field 10/20/2026). Used in Output_Manager rung 2.
  v6g: slow-down AND stop decel proportional to the rewinder diameter.
       The ATV930 ramp is in Hz/s (motor = rewind shaft), so the web
       decel [mm/s2] = shaft decel x radius. Field data 10/20/2026:
       24 mm/s2 at 120 mm -> 90 mm/s2 at 600 mm.
       Learned: RwSlowDecelK, RwStopDecelK [mm/s2 per mm of diameter].
       Live:    RwSlowDecelLrn [mm/s2], RwStopDecel [pulses/s2] = K x DecelDiam.
  v6f: learn the REAL slow-down deceleration (section 5b) -> RwSlowDecelLrn
       (mm/s2). The ATV930 stretches its ramp with a heavy roll (no
       braking resistor), so the PLC ramp (DecelCoarse) was too
       optimistic: the stop came before final speed was reached.
       Priority in section 6: RwSlowDecel (manual) > RwSlowDecelLrn
       (learned) > DecelCoarse/DecelTrim model.
  v6e: GSM_Learned_OK (replaces global RwLearnFault): TRUE only when
       RwMat_K is learned, RwStopDecel is learned (if RwLearnStop) and
       there is no learning fault. Alarm 16 uses GSM_LearnFault.
  v6d: exact 100 ms tick (25 x 4 ms scan counter, the self-resetting
       TON ran ~104-108 ms -> speed ~4-8 % too high)
     standstill = <= 2 counts / 100 ms (encoder jitter)
     stop learning skipped after jog / manual / slow-button runs and
       when the auxiliaries drop (E-stop): RunWasJog, StopLearnValid
     work / final speed from the potentiometers in BOTH modes
       (Speed_Display / LowSpd_Display are not m/min in ManMode)
     slow-down length includes the DecelTrim tail inside the
       SpdDelta_Percent band (Motor_Manager rungs 14/15)
  v6c: learning supervision -> RwLearnFault (alarm 16), learned paper
       thickness RwThick_mm, plausibility limits for K and stop decel
  v6b: RwMaxSlowAtDiam display + RwMaxSlowFallback (slow-down warning
```

```
56          while RwMat_K is not learned yet)
57      v6 : consistent internal names, units in names.
58          Globals renamed: RwRemain_m, RwStopLen_m, RwSlowLen_m,
59                      RwSlowMargin_m, UwSlowMargin_m
60      v5 : max diameter protection (always active), sensor plausibility,
61          12SA5 master (section 9), alarms in section Alarms (ladder)
62      v4b: AUTO interlock + single HMI display (SlowAtDisp / SlowAtMode)
63
    ================================================================
    ==================== *)
64
65  (* ---- 1. 100 ms tick, mm per encoder pulse ------------------------------ *)
66  (* v6d: 25 scans x 4 ms PrimaryTask = exactly 100 ms.                    *)
67  (*      Change 25 if the PrimaryTask period is changed.                  *)
68  TickCnt := TickCnt + 1;
69  IF TickCnt >= 25 THEN
70      TickCnt := 0;
71      Tick := TRUE;
72  ELSE
73      Tick := FALSE;
74  END_IF;
75
76  IF (Encoder_Pulse > 0) AND (EncoderWheel_Diam > 0.0) THEN
77      MmPerPulse := EncoderWheel_Diam / (INT_TO_REAL(Encoder_Pulse) * 4.0);
78  ELSE
79      MmPerPulse := 0.0;
80  END_IF;
81
82  (* ---- 2. Rewinder DT35 -> filtered distance -> diameter ----------------- *)
83  RwDistRaw_mm := RwDist0V + (INT_TO_REAL(AI_Diam_Rewinder) / 4000.0)
84              * (RwDist10V - RwDist0V);
85
86  IF Tick THEN
87      IF NOT RwFiltInit THEN
88          RwDist := RwDistRaw_mm;
89          RwFiltInit := TRUE;
90      ELSE
91          RwDist := RwDist + RwFilter_K * (RwDistRaw_mm - RwDist);
92      END_IF;
93  END_IF;
94
95  RwSensFault := (AI_Diam_Rewinder < RwRawMin) OR (AI_Diam_Rewinder >
    RwRawMax)
96              OR (RwDist > RwAxisDist);
97
98  RwDiam := 2.0 * (RwAxisDist - RwDist);
99  IF RwDiam < 0.0 THEN
100     RwDiam := 0.0;
101 END_IF;
102 (* RwAxisDist = DT35 reference to rewind shaft centre (mm), set manually *)
103
104 (* v6g: diameter used by the decel model (last valid on sensor fault)     *)
105 IF NOT RwSensFault THEN
106     DecelDiam := RwDiam;
107 ELSE
108     DecelDiam := RwLastValidDiam;
109 END_IF;
110 IF DecelDiam < 100.0 THEN
111     DecelDiam := 100.0;                    (* core *)
```

```
112 END_IF;
113
114 (* ---- 3. Actual speed from encoder, 100 ms window ---------------------- *)
115 IF Tick THEN
116     IF NOT SpdInit THEN
117         EncPrev := Encoder_Pos;
118         SpdInit := TRUE;
119     END_IF;
120     SpdAct_Pps := DINT_TO_REAL(ABS(Encoder_Pos - EncPrev)) * 10.0;
121     EncPrev := Encoder_Pos;
122 END_IF;
123 RwSpeed := SpdAct_Pps * MmPerPulse * 0.06;          (* m/min *)
124
125 (* ---- 4. Learn material factor  K = dDiam/dPulse * Diam ----------------- *)
126 IF Tick THEN
127     LearnWinTicks := LearnWinTicks + 1;
128     IF NOT LearnInit THEN
129         LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
130         LearnWinTicks := 0;
131         LearnWinRunning := O_MotorStart;
132         LearnInit := TRUE;
133     ELSE
134         LearnWinRunning := LearnWinRunning AND O_MotorStart;
135         IF LearnWinTicks >= RwWinTicks THEN
136             LearnWin_Pls := ABS(Encoder_Pos - LearnWinStartPos);
137             LearnWinDiamDelta := RwDiam - LearnWinStartDiam;
138             (* A window SHOULD learn: ran all the time, enough length.  *)
139             (* DT35 OK. Then it must give a plausible paper thickness. *)
140             IF LearnWinRunning AND (LearnWin_Pls >= RwMinPulses)
141               AND NOT RwSensFault THEN
142                 LearnWinOk := FALSE;
143                 IF (LearnWinDiamDelta > 0.0) AND (MmPerPulse > 0.0) THEN
144                     MatK_Window := (LearnWinDiamDelta / DINT_TO_REAL(LearnWin_Pls))
145                                 * ((RwDiam + LearnWinStartDiam) / 2.0);
146                     (* K = 2 * thickness * mm-per-pulse / PI          *)
147                     ThickWin_mm := MatK_Window * 3.14159 / (2.0 * MmPerPulse);
148                     IF (ThickWin_mm >= RwThickMin_mm)
149                       AND (ThickWin_mm <= RwThickMax_mm) THEN
150                         LearnWinOk := TRUE;
151                         IF RwMat_K <= 0.0 THEN
152                             RwMat_K := MatK_Window;
153                         ELSE
154                             RwMat_K := RwMat_K + RwLearn_K * (MatK_Window - RwMat_K);
155                         END_IF;
156                     END_IF;
157                 END_IF;
158                 IF LearnWinOk THEN
159                     LearnFailCnt := 0;
160                 ELSIF LearnFailCnt < 1000 THEN
161                     LearnFailCnt := LearnFailCnt + 1;  (* no growth / implausible *)
162                 END_IF;
163             END_IF;
164             LearnWinStartPos := Encoder_Pos;  LearnWinStartDiam := RwDiam;
165             LearnWinTicks := 0;
166             LearnWinRunning := O_MotorStart;
167         END_IF;
168     END_IF;
169 END_IF;
170
```

```
171 (* ---- 5. Learn real stop deceleration on every stop ---------- *)
172 (* v6d: a run with jog / manual / slow button uses the 2nd ramp (O_AccDec2) *)
173 (*      -> its stop must not be learned. Latched for the whole run because  *)
174 (*      I_LowSpd_OS is already released on the stop edge.                    *)
175 IF O_MotorStart AND NOT MotorRunMem THEN          (* start edge *)
176     RunWasJog := FALSE;
177 END_IF;
178 IF O_MotorStart AND (ManMode OR I_LowSpd_OS) THEN
179     RunWasJog := TRUE;
180 END_IF;
181
182 IF MotorRunMem AND NOT O_MotorStart THEN          (* stop command edge *)
183     StopMeasActive := TRUE;
184     StopStartPos := Encoder_Pos;
185     StopStartSpd_Pps := SpdAct_Pps;
186     StopStartDiam := DecelDiam;
187     StandstillTicks := 0;
188     (* STOP button and automatic stop take the same path (drive DI1 and    *)
189     (* O_Brake -> T-one drop together, electrical plan sheets 1/9/15),     *)
190     (* so both are learned.                                                *)
191     StopLearnValid := ControlVoltage AND NOT RunWasJog AND NOT ManMode;
192 END_IF;
193 MotorRunMem := O_MotorStart;
194
195 IF O_MotorStart THEN
196     StopMeasActive := FALSE;                  (* restarted: abort *)
197 ELSIF StopMeasActive AND Tick THEN
198     IF NOT ControlVoltage THEN
199         StopLearnValid := FALSE;              (* E-stop during the stop *)
200     END_IF;
201     IF SpdAct_Pps <= 20.0 THEN                (* <= 2 counts / 100 ms *)
202         StandstillTicks := StandstillTicks + 1;
203     ELSE
204         StandstillTicks := 0;
205     END_IF;
206     IF StandstillTicks >= 3 THEN              (* standstill 300 ms *)
207         StopMeasActive := FALSE;
208         StopMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - StopStartPos));
209         IF RwLearnStop AND StopLearnValid
210           AND (StopStartSpd_Pps * MmPerPulse * 0.06 >= RwMinLearnSpd) THEN
211           (* This stop SHOULD learn: result must be plausible       *)
212           StopLearnOk := FALSE;
213           IF StopMeas_Pls > StopStartSpd_Pps * RwTreact THEN
214               StopDecelMeas := (StopStartSpd_Pps * StopStartSpd_Pps)
215                             / (2.0 * (StopMeas_Pls - StopStartSpd_Pps * RwTreact));
216               StopDecelMeas_mm := StopDecelMeas * MmPerPulse;  (* mm/s2 *)
217               IF (StopDecelMeas_mm >= RwDecelMin_mm)
218                 AND (StopDecelMeas_mm <= RwDecelMax_mm) THEN
219                   StopLearnOk := TRUE;
220                   (* v6g: learn per mm of diameter *)
221                   DecelK_Meas := StopDecelMeas_mm / ((StopStartDiam + DecelDiam) /
    2.0);
222                   IF RwStopDecelK <= 0.0 THEN
223                       RwStopDecelK := DecelK_Meas;
224                   ELSE
225                       RwStopDecelK := RwStopDecelK + 0.3 * (DecelK_Meas -
    RwStopDecelK);
226                   END_IF;
227               END_IF;
```

```
228          END_IF;
229          IF StopLearnOk THEN
230             StopFailCnt := 0;
231          ELSIF StopFailCnt < 1000 THEN
232             StopFailCnt := StopFailCnt + 1;
233          END_IF;
234       END_IF;
235    END_IF;
236 END_IF;
237
238 (* ---- 5b. v6f: learn real slow-down deceleration ----------------------- *)
239 (* From the SlowDown edge (at speed) until final speed is reached, or     *)
240 (* until the stop comes first. a = (v0^2 - v1^2) / (2 * distance).        *)
241 (* Not learned in jog / manual / slow button (2nd ramp).                  *)
242 IF SlowDown AND NOT SlowMem AND O_MotorStart
243    AND NOT ManMode AND NOT I_LowSpd_OS
244    AND (SpdAct_Pps > SpdFinal_Pps + SpdBand_Pps) THEN
245    SlowMeasActive := TRUE;
246    SlowStartPos := Encoder_Pos;
247    SlowStartSpd_Pps := SpdAct_Pps;
248    SlowStartDiam := DecelDiam;
249 END_IF;
250 SlowMem := SlowDown;
251
252 IF SlowMeasActive THEN
253    IF NOT SlowDown OR ManMode OR I_LowSpd_OS THEN
254       SlowMeasActive := FALSE;                  (* aborted *)
255    ELSIF Tick AND ((SpdAct_Pps <= SpdFinal_Pps + 0.25 * SpdBand_Pps)
256                OR NOT O_MotorStart) THEN       (* final reached or stopped *)
257       SlowMeasActive := FALSE;
258       SlowMeas_Pls := DINT_TO_REAL(ABS(Encoder_Pos - SlowStartPos));
259       IF (SlowMeas_Pls > 0.0) AND (MmPerPulse > 0.0)
260          AND ((SlowStartSpd_Pps - SpdAct_Pps) * MmPerPulse * 0.06 >= 10.0) THEN
261          SlowDecelMeas_mm := (SlowStartSpd_Pps * SlowStartSpd_Pps
262                             - SpdAct_Pps * SpdAct_Pps)
263                            / (2.0 * SlowMeas_Pls) * MmPerPulse;  (* mm/s2 *)
264          IF (SlowDecelMeas_mm >= RwDecelMin_mm)
265             AND (SlowDecelMeas_mm <= RwDecelMax_mm) THEN
266             (* v6g: learn per mm of diameter *)
267             DecelK_Meas := SlowDecelMeas_mm / ((SlowStartDiam + DecelDiam) /
    2.0);
268             IF RwSlowDecelK <= 0.0 THEN
269                RwSlowDecelK := DecelK_Meas;
270             ELSE
271                RwSlowDecelK := RwSlowDecelK + 0.5 * (DecelK_Meas -
    RwSlowDecelK);
272             END_IF;
273          END_IF;
274       END_IF;
275    END_IF;
276 END_IF;
277
278 (* ---- 5c. v6g: live decels = learned K x current diameter -------------- *)
279 (* One-time seed of RwStopDecelK from the v6f value (first scan after the  *)
280 (* transfer). To re-learn: write 0 to RwStopDecelK AND RwStopDecel.        *)
281 IF (RwStopDecelK <= 0.0) AND (RwStopDecel > 0.0) AND (MmPerPulse > 0.0) THEN
282    RwStopDecelK := RwStopDecel * MmPerPulse / DecelDiam;
283 END_IF;
284 IF (RwStopDecelK > 0.0) AND (MmPerPulse > 0.0) THEN
```

```
    RwStopDecel := RwStopDecelK * DecelDiam / MmPerPulse;     (* pulses/s2 *)
END_IF;
IF RwSlowDecelK > 0.0 THEN
    RwSlowDecelLrn := RwSlowDecelK * DecelDiam;               (* mm/s2 *)
ELSE
    RwSlowDecelLrn := 0.0;
END_IF;

(* ---- 6. Lengths in pulses: stop, slow-down, remaining ------------------ *)
(* Stop length uses the ACTUAL speed (precision)                          *)
IF RwStopDecel > 0.0 THEN
    StopLen_Pls := SpdAct_Pps * RwTreact
                + (SpdAct_Pps * SpdAct_Pps) / (2.0 * RwStopDecel);
ELSE
    StopLen_Pls := SpdAct_Pps * (RwTreact + RwTstopDef / 2.0);
END_IF;
StopLen_Pls := StopLen_Pls * RwTrim;

(* Slow-down uses SpdRef = MAX(actual, work-speed setpoint):              *)
(* stable value, shown to the operator even before start                  *)
SlowLenBase_Pls := 0.0;  RwSlowLen_Pls := 0.0;  UwSlowLen_Pls := 0.0;
IF MmPerPulse > 0.0 THEN
    (* v6f: manual > learned > PLC ramp model *)
    IF RwSlowDecel > 0.0 THEN
        SlowDecel_Pps2 := RwSlowDecel / MmPerPulse;
        SlowOneRamp := TRUE;
    ELSIF RwSlowDecelLrn > 0.0 THEN
        SlowDecel_Pps2 := RwSlowDecelLrn / MmPerPulse;  (* real, incl. trim tail *)
        SlowOneRamp := TRUE;
    ELSE
        SlowDecel_Pps2 := DecelCoarse * 250.0 / MmPerPulse; (* 1 = 250 mm/s2 *)
        SlowOneRamp := FALSE;
    END_IF;
    (* v6d: DecelTrim ramp used inside the +/- SpdDelta_Percent band *)
    IF DecelTrim > 0.0 THEN
        TrimDecel_Pps2 := DecelTrim * 250.0 / MmPerPulse;
    ELSE
        TrimDecel_Pps2 := SlowDecel_Pps2;
    END_IF;
    SpdBand_Pps := SpdDelta_Percent * 1000.0 / 60.0 / MmPerPulse;

    (* v6d: speeds from the potentiometers, valid in auto AND ManMode     *)
    (*      (m/min = Pot / Input_Ratio, same as Motor_Manager rung 8)     *)
    IF (Input_Ratio > 0.0) AND ManMode THEN
        (* v6i: ManMode drives the inverter with the raw pots (rung 17) *)
        SpdFinal_Pps := INT_TO_REAL(Pot2_Slow_Velocity) / Input_Ratio * 1000.0 /
60.0 / MmPerPulse;
        SpdWork_Pps  := INT_TO_REAL(Pot1_Work_Velocity) / Input_Ratio * 1000.0 /
60.0 / MmPerPulse;
    ELSIF Input_Ratio > 0.0 THEN
        SpdFinal_Pps := INT_TO_REAL(Pot2_Mem) / Input_Ratio * 1000.0 / 60.0 /
MmPerPulse;
        SpdWork_Pps  := INT_TO_REAL(Pot1_Mem) / Input_Ratio * 1000.0 / 60.0 /
MmPerPulse;
    ELSE
        SpdFinal_Pps := LowSpd_Display * 1000.0 / 60.0 / MmPerPulse;
        SpdWork_Pps  := Speed_Display  * 1000.0 / 60.0 / MmPerPulse;
    END_IF;
    SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
```

```
339     SpdRef_Pps := MAX(SpdAct_Pps, SpdWork_Pps);
340
341     IF RwStopDecel > 0.0 THEN
342         StopLenFinal_Pls := SpdFinal_Pps * RwTreact
343                             + (SpdFinal_Pps * SpdFinal_Pps) / (2.0 * RwStopDecel);
344     ELSE
345         StopLenFinal_Pls := SpdFinal_Pps * (RwTreact + RwTstopDef / 2.0);
346     END_IF;
347
348     SlowLenBase_Pls := StopLenFinal_Pls * RwTrim;
349     IF (SpdRef_Pps > SpdFinal_Pps) AND (SlowDecel_Pps2 > 0.0) THEN
350         IF SlowOneRamp OR (TrimDecel_Pps2 <= 0.0) THEN
351             (* manual or learned decel: one constant ramp *)
352             SlowLenBase_Pls := SlowLenBase_Pls
353                 + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
354                  / (2.0 * SlowDecel_Pps2);
355         ELSE
356             (* auto: coarse ramp down to the band, trim ramp inside it *)
357             SpdKnee_Pps := SpdFinal_Pps + SpdBand_Pps;
358             IF SpdRef_Pps > SpdKnee_Pps THEN
359                 SlowLenBase_Pls := SlowLenBase_Pls
360                     + (SpdRef_Pps * SpdRef_Pps - SpdKnee_Pps * SpdKnee_Pps)
361                      / (2.0 * SlowDecel_Pps2)
362                     + (SpdKnee_Pps * SpdKnee_Pps - SpdFinal_Pps * SpdFinal_Pps)
363                      / (2.0 * TrimDecel_Pps2);
364             ELSE
365                 SlowLenBase_Pls := SlowLenBase_Pls
366                     + (SpdRef_Pps * SpdRef_Pps - SpdFinal_Pps * SpdFinal_Pps)
367                      / (2.0 * TrimDecel_Pps2);
368             END_IF;
369         END_IF;
370     END_IF;
371
372     RwSlowLen_Pls := SlowLenBase_Pls + RwSlowMargin_m * 1000.0 / MmPerPulse;
373
374     IF UwSlowMargin_m > 0.0 THEN
375         UwMarginUsed_m := UwSlowMargin_m;
376     ELSE
377         UwMarginUsed_m := RwSlowMargin_m;
378     END_IF;
379     UwSlowLen_Pls := SlowLenBase_Pls + UwMarginUsed_m * 1000.0 / MmPerPulse;
380 END_IF;
381
382 IF RwDiam >= RwTargetDiam THEN
383     RwRemain_Pls := 0.0;
384 ELSIF RwMat_K > 0.0 THEN
385     RwRemain_Pls := (RwTargetDiam * RwTargetDiam - RwDiam * RwDiam)
386                     / (2.0 * RwMat_K);
387 ELSE
388     RwRemain_Pls := 1.0E9;                    (* K not learned: plain target *)
389 END_IF;
390
391 RwRemain_m  := RwRemain_Pls  * MmPerPulse / 1000.0;
392 RwStopLen_m := StopLen_Pls   * MmPerPulse / 1000.0;
393 RwSlowLen_m := RwSlowLen_Pls * MmPerPulse / 1000.0;
394
395 (* ---- 6b. Live operator displays ---------------------------------------- *)
396 (* Rewinder: diameter where slow-down / stop command will happen          *)
397 IF RwMat_K > 0.0 THEN
```

```
398     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
399     IF SqrtArg > 0.0 THEN
400         RwSlowAtDiam := SQRT(SqrtArg);
401     ELSE
402         RwSlowAtDiam := 0.0;
403     END_IF;
404     SqrtArg := RwTargetDiam * RwTargetDiam - 2.0 * RwMat_K * StopLen_Pls;
405     IF SqrtArg > 0.0 THEN
406         RwStopAtDiam := SQRT(SqrtArg);
407     ELSE
408         RwStopAtDiam := 0.0;
409     END_IF;
410 ELSE
411     RwSlowAtDiam := RwTargetDiam;              (* not learned yet *)
412     RwStopAtDiam := RwTargetDiam;
413 END_IF;
414
415 (* Unwinder: meter count where the slow-down will start              *)
416 IF MmPerPulse > 0.0 THEN
417     UwSlow_at_Metre := UwStop_At_Meter - UwSlowLen_Pls * MmPerPulse / 1000.0;
418 ELSE
419     UwSlow_at_Metre := UwStop_At_Meter;
420 END_IF;
421 IF UwSlow_at_Metre < 0.0 THEN
422     UwSlow_at_Metre := 0.0;
423 END_IF;
424
425 (* Unwinder slow-down request, latched                              *)
426 IF (Current_Metre < UwSlow_at_Metre - 50.0) OR (UwStop_At_Meter <= 0.0) THEN
427     UwSlowReq := FALSE;                    (* counter reset / longer roll *)
428 END_IF;
429 IF (UwStop_At_Meter > 0.0) AND (Current_Metre >= UwSlow_at_Metre) THEN
430     UwSlowReq := TRUE;
431 END_IF;
432
433 (* ---- 7. Rewinder requests: latched, reset when a new core is loaded ---- *)
434 IF NOT RwStopEnable
435    OR ((RwDiam < RwTargetDiam - RwResetHyst) AND NOT RwSensFault) THEN
436     RwStopReq := FALSE;
437     RwSlowReq := FALSE;
438 END_IF;
439
440 IF RwStopEnable THEN
441     IF (RwRemain_Pls <= StopLen_Pls) OR (RwSensFault AND RwStopOnFault) THEN
442         RwStopReq := TRUE;
443     END_IF;
444     IF RwUseSlow AND (RwRemain_Pls <= RwSlowLen_Pls) THEN
445         RwSlowReq := TRUE;
446     END_IF;
447 END_IF;
448
449 (* ---- 8. One AUTO at a time + single "SLOW DOWN AT" display ---------- *)
450 IF UwStopEnable AND NOT UwEnMem THEN              (* unwinder AUTO switched
    ON *)
451     RwStopEnable := FALSE;
452 END_IF;
453 IF RwStopEnable AND NOT RwEnMem THEN              (* rewinder AUTO switched
    ON *)
454     UwStopEnable := FALSE;
```

```
455 END_IF;
456 UwEnMem := UwStopEnable;
457 RwEnMem := RwStopEnable;
458
459 IF RwStopEnable THEN
460     SlowAtDisp := RwSlowAtDiam;                (* mm *)
461     SlowAtMode := 2;
462 ELSIF UwStopEnable THEN
463     SlowAtDisp := UwSlow_at_Metre;             (* m  *)
464     SlowAtMode := 1;
465 ELSE
466     SlowAtDisp := 0.0;
467     SlowAtMode := 0;                       (* hide display *)
468 END_IF;
469 SlowAtShow := (SlowAtMode <> 0);
470
471 (* ---- 9. Automatic stop master = selector 12SA5 (I_AutoStop_OS) -------- *)
472 AutoSlowCmd := I_AutoStop_OS AND
473             ((UwStopEnable AND UwSlowReq) OR (RwStopEnable AND RwSlowReq));
474
475 AutoStopCmd := I_AutoStop_OS AND
476             ((UwStopEnable AND (UwStop_At_Meter > 0.0)
477               AND (Current_Metre >= UwStop_At_Meter))
478              OR (RwStopEnable AND RwStopReq));
479
480 (* ---- 10. Unwinder roll empty (for alarm 15) ---------------------------- *)
481 IF UwDiam_Act > (UwDiamStop + UwEmptyHyst) THEN
482     UnWinder_Empty := FALSE;                   (* new roll loaded *)
483 END_IF;
484 IF (UwDiam_Act > 0.0) AND (UwDiam_Act <= UwDiamStop) THEN
485     UnWinder_Empty := TRUE;                    (* roll empty *)
486 END_IF;
487
488 (* ---- 11. Rewinder max diameter protection - ALWAYS active ------------- *)
489 (* Independent of selector 12SA5 and of the AUTO buttons.              *)
490 (* Mechanical limit 680 mm radius = 1360 mm                           *)
491 IF NOT RwSensFault THEN
492     RwLastValidDiam := RwDiam;
493 END_IF;
494
495 IF RwDiam >= RwMaxDiam THEN
496     RwRemainMax_Pls := 0.0;
497 ELSIF RwMat_K > 0.0 THEN
498     RwRemainMax_Pls := (RwMaxDiam * RwMaxDiam - RwDiam * RwDiam)
499                        / (2.0 * RwMat_K);
500 ELSE
501     RwRemainMax_Pls := 1.0E9;                  (* K not learned: plain limit *)
502 END_IF;
503
504 (* Display: diameter where the max-protection slow-down will start        *)
505 IF RwMat_K > 0.0 THEN
506     SqrtArg := RwMaxDiam * RwMaxDiam - 2.0 * RwMat_K * RwSlowLen_Pls;
507     IF SqrtArg > 0.0 THEN
508         RwMaxSlowAtDiam := SQRT(SqrtArg);
509     ELSE
510         RwMaxSlowAtDiam := 0.0;
511     END_IF;
512 ELSIF RwMaxSlowFallback > 0.0 THEN
513     RwMaxSlowAtDiam := RwMaxSlowFallback;      (* fallback in use *)
```

```
514  ELSE
515      RwMaxSlowAtDiam := RwMaxDiam;
516  END_IF;
517
518 (* Reset only after the roll is removed (diameter well below the limit)    *)
519 IF (RwDiam < RwMaxDiam - RwResetHyst) AND NOT RwSensFault THEN
520      RwMaxStop := FALSE;
521      RwMaxSlow := FALSE;
522  END_IF;
523
524 IF NOT RwSensFault THEN
525      IF RwRemainMax_Pls <= StopLen_Pls THEN
526          RwMaxStop := TRUE;                    (* speed-compensated stop *)
527      END_IF;
528      IF RwRemainMax_Pls <= RwSlowLen_Pls THEN
529          RwMaxSlow := TRUE;                    (* warning slow-down + horn *)
530      END_IF;
531      IF (RwMat_K <= 0.0) AND (RwMaxSlowFallback > 0.0)
532        AND (RwDiam >= RwMaxSlowFallback) THEN
533          RwMaxSlow := TRUE;                    (* fallback while K not learned *)
534      END_IF;
535  ELSIF (AI_Diam_Rewinder < RwRawMin) AND (RwLastValidDiam > RwMaxDiam -
     100.0) THEN
536      RwMaxStop := TRUE;                        (* roll grew into the DT35 near zone *)
537  END_IF;
538
539 (* Max diameter protection added OUTSIDE the 12SA5 master (section 9)      *)
540  AutoSlowCmd := AutoSlowCmd OR RwMaxSlow;
541  AutoStopCmd := AutoStopCmd OR RwMaxStop;
542
543 (* ---- 12. Learning supervision -> alarm 16 (section Alarms) ------------ *)
544 (* Learned paper thickness, for display / plausibility check              *)
545 IF MmPerPulse > 0.0 THEN
546      RwThick_mm := RwMat_K * 3.14159 / (2.0 * MmPerPulse);
547  ELSE
548      RwThick_mm := 0.0;
549  END_IF;
550
551 (* Fault: too many consecutive windows / stops that should have           *)
552 (* learned but did not. Clears by itself on the next good result.        *)
553  GSM_LearnFault := (LearnFailCnt >= RwLearnFailMax)
554              OR (StopFailCnt >= RwStopFailMax);      (* -> alarm 16 *)
555
556 (* v6e: learning status for the HMI. FALSE until learned.                 *)
557  GSM_Learned_OK := (RwMat_K > 0.0)
558                AND ((RwStopDecel > 0.0) OR NOT RwLearnStop)
559                AND NOT GSM_LearnFault;
560
561 (* ---- 13. v6h: keep T-one in automatic tension control during the stop - *)
562 (* While running: BrakeHold = Brake_Cmd (no change in running behaviour).  *)
563 (* After the stop: stays ON until the web is below 5 m/min, after 15 s,    *)
564 (* or on E-stop. Already TRUE on the stop scan (Output_Manager runs before *)
565 (* this section), so O_Brake never drops for one scan.                    *)
566 (* Output_Manager rung 2:  O_Brake := Brake_Cmd OR BrakeHold              *)
567 IF O_MotorStart THEN
568      BrakeHold := Brake_Cmd AND ControlVoltage;
569      BrakeHoldTicks := 0;
570  ELSIF BrakeHold THEN
571      IF Tick THEN
```

```
572         BrakeHoldTicks := BrakeHoldTicks + 1;
573       END_IF;
574       IF NOT ControlVoltage OR (RwSpeed < 5.0) OR (BrakeHoldTicks >= 150) THEN
575         BrakeHold := FALSE;
576       END_IF;
577     END_IF;
578
579   (* ---- 14. v6j: TEACH CORE - automatic offset with an empty core -------- *)
580   (* HMI: set RwTeachCore (button, Set ON). Conditions: motor stopped, web  *)
581   (* not moving, DT35 OK. After 2 s stable (filter settled):               *)
582   (*    RwAxisDist := RwDist + RwCoreOD / 2   -> RwDiam = RwCoreOD         *)
583   (* RwTeachState: 0 idle, 1 teaching (wait 2 s), 2 done OK,               *)
584   (*               3 refused (motor running / web moving / DT35 fault).    *)
585   (* Only the offset is taught: the scale is RwDist10V - RwDist0V          *)
586   (* (= real distance between the two DT35 taught points).                 *)
587   IF RwTeachCore THEN
588     IF O_MotorStart OR (RwSpeed > 1.0) OR RwSensFault
589       OR NOT RwFiltInit OR (RwCoreOD <= 0.0) THEN
590       RwTeachState := 3;                    (* refused *)
591       RwTeachCore := FALSE;
592       TeachTicks := 0;
593     ELSE
594       RwTeachState := 1;                    (* teaching *)
595       IF Tick THEN
596         TeachTicks := TeachTicks + 1;
597       END_IF;
598       IF TeachTicks >= 20 THEN              (* 2 s stable *)
599         RwAxisDist_Prev := RwAxisDist;      (* v6l: keep previous *)
600         RwAxisDist := RwDist + RwCoreOD / 2.0;
601         RwTeachState := 2;                  (* done *)
602         RwTeachCore := FALSE;
603         TeachTicks := 0;
604       END_IF;
605     END_IF;
606   ELSE
607     TeachTicks := 0;
608     IF O_MotorStart THEN
609       RwTeachState := 0;                    (* clear status on next run *)
610     END_IF;
611   END_IF;
612
613   (* ---- 14b. v6k: unwinder TEACH CORE (HMI button Core_Measure) ---------- *)
614   (* Same behaviour as section 14. Sonar is not filtered, so the 2 s are   *)
615   (* averaged:  UwCoreDistance := average(UwDiam_Measure)  (sonar -> core   *)
616   (* surface). UwDiam_Act = (UwCoreDistance - UwDiam_Measure) * 2          *)
617   (*                      + UwCoreDiameter   (General rung 14, unchanged). *)
618   (* UwTeachState: 0 idle, 1 teaching, 2 done OK, 3 refused.               *)
619   IF Core_Measure THEN
620     IF O_MotorStart OR (RwSpeed > 1.0) OR (UwCoreDiameter <= 0.0)
621       OR (UwDiam_Measure <= 0.0) THEN
622       UwTeachState := 3;                    (* refused *)
623       Core_Measure := FALSE;
624       UwTeachTicks := 0;
625       UwTeachSum := 0.0;
626     ELSE
627       UwTeachState := 1;                    (* teaching *)
628       IF Tick THEN
629         UwTeachTicks := UwTeachTicks + 1;
630         UwTeachSum := UwTeachSum + UwDiam_Measure;
```

```
631         END_IF;
632         IF UwTeachTicks >= 20 THEN              (* 2 s, 20 samples *)
633           UwCoreDist_Prev := UwCoreDistance;    (* v6l: keep previous *)
634           UwCoreDistance := UwTeachSum / INT_TO_REAL(UwTeachTicks);
635           UwTeachState := 2;                    (* done *)
636           Core_Measure := FALSE;
637           UwTeachTicks := 0;
638           UwTeachSum := 0.0;
639         END_IF;
640       END_IF;
641     ELSE
642       UwTeachTicks := 0;
643       UwTeachSum := 0.0;
644       IF O_MotorStart THEN
645         UwTeachState := 0;                      (* clear status on next run *)
646       END_IF;
647     END_IF;
648
649   (* ---- 14c. v6l: UNDO teach = swap current <-> previous (machine stopped) *)
650   (* Pressing UNDO twice goes back to the taught value. State 4 = RESTORED. *)
651   IF RwTeachUndo THEN
652     IF NOT O_MotorStart AND (RwAxisDist_Prev > 0.0) THEN
653       TeachSwap := RwAxisDist;
654       RwAxisDist := RwAxisDist_Prev;
655       RwAxisDist_Prev := TeachSwap;
656       RwTeachState := 4;                        (* restored *)
657     ELSE
658       RwTeachState := 3;                        (* refused *)
659     END_IF;
660     RwTeachUndo := FALSE;
661   END_IF;
662   IF UwTeachUndo THEN
663     IF NOT O_MotorStart AND (UwCoreDist_Prev > 0.0) THEN
664       TeachSwap := UwCoreDistance;
665       UwCoreDistance := UwCoreDist_Prev;
666       UwCoreDist_Prev := TeachSwap;
667       UwTeachState := 4;                        (* restored *)
668     ELSE
669       UwTeachState := 3;                        (* refused *)
670     END_IF;
671     UwTeachUndo := FALSE;
672   END_IF;
```

1-8.Data

1-8-1.Data Types

Name	Type	Comment

1-8-2.Global Variables

Name	Data Type	Initial Value	AT	Retain	Constant	Network Publish	Comment
VAR_GLOBAL							
Pot1_Work_Velocity	INT		BuiltInIO://opt/#0/Ch1_Analog_Input_Value	False	False	Publish Only	Potentiometer 1 value - Work speed -
InvRef	INT		BuiltInIO://opt/#0/Ch2_Analog_Output_Value	False	False	Publish Only	Inverter speed reference
Fix10Vdc	INT		BuiltInIO://opt/#0/Ch1_Analog_Output_Value	False	False	Publish Only	Fixed 10 Vdc generator
VelMax	REAL	800		True	False	Publish Only	Maximum speed
VelSet	REAL			False	False	Publish Only	Set speed
RwSlowDecelK	REAL	0.15		True	False	Publish Only	Learned slow-down decel per mm of rewinder diameter (mm/s2 per mm)
RwMaxSlowFallback	REAL	1310.0		True	False	Publish Only	Rewinder max: slow-down diameter while K not learned (mm)
RwThickMin_mm	REAL	0.02		True	False	Publish Only	Learning: min plausible paper thickness (mm)
RwThickMax_mm	REAL	2.0		True	False	Publish Only	Learning: max plausible paper thickness (mm)
RwDecelMin_mm	REAL	20.0		True	False	Publish Only	Learning: min plausible stop decel (mm/s2)
RwDecelMax_mm	REAL	5000.0		True	False	Publish Only	Learning: max plausible stop decel (mm/s2)
RwLearnFailMax	INT	12		True	False	Publish Only	Learning: failed windows before alarm (12 = 2 min)
RwStopFailMax	INT	3		True	False	Publish Only	Learning: failed stops before alarm
RwThick_mm	REAL			False	False	Publish Only	Learned paper thickness (mm)
OverSpeed	BOOL			False	False	Publish Only	Speed too high
WorkSpd_Max	REAL	400.0		True	False	Publish Only	Work speed pot full scale (m/min), 50..VelMax
WorkSpd_Max_Set	REAL	400.0		True	False	Publish Only	HMI request: work pot full scale (m/min), applied only with motor stopped
FinalSpd_Max_Set	REAL	40.0		True	False	Publish Only	HMI request: final pot full scale (m/min), applied only with motor stopped
FinalSpd_Max	REAL	40.0		True	False	Publish Only	Final speed pot full scale (m/min), 5..WorkSpd_Max
UnderSpeed	BOOL			False	False	Publish Only	Speed too low
RwCoreOD	REAL	100.0		True	False	Publish Only	Rewinder core outside diameter for TEACH CORE (mm)
RwTeachCore	BOOL	False		False	False	Publish Only	HMI: TEACH CORE request (PLC resets it)
RwTeachState	INT	0		False	False	Publish Only	TEACH CORE: 0 idle 1 teaching 2 done 3 refused
SpdDelta_Percent	REAL			True	False	Publish Only	Speed difference percentage
Encoder_Pos	DINT		IOBus://unit#3/Ch1 Encoder Pres	False	False	Publish Only	Encoder counter

			ent Position				
UwTeachState	INT	0		False	False	Publish Only	Unwinder TEACH CORE: 0 idle 1 teaching 2 done 3 refused
PulseRate	UDINT		IOBus://unit#3/Ch1 Pulse Rate	False	False	Publish Only	Pulses per 100 ms
Encoder_On	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Counter Enable	False	False	Publish Only	Encoder enable
Encoder_Reset	BOOL		IOBus://unit#3/Ch1 Encoder Counter Operation Command/Ch1 Internal Reset Execution	False	False	Publish Only	Encoder reset
HMI_Screen	ARRAY[0..1] OF DINT			False	False	Publish Only	HMI panel Screen
Velocity_M_min	REAL			True	False	Publish Only	Speed in m/min
Current_Metre	REAL			True	False	Publish Only	Current metres
I_RsetCnt	BOOL			False	False	Publish Only	Metre counter reset button
Speed_Display	REAL			True	False	Publish Only	Set speed (displayed)
I_ControlVoltage	BOOL		BuiltInIO://cpu/#0/Input_Bit_00	False	False	Publish Only	Auxiliaries ON
I_Start_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_01	False	False	Publish Only	Start button
I_Stop_PB	BOOL		BuiltInIO://cpu/#0/Input_Bit_02	False	False	Publish Only	Stop button
I_Motor_Thermal	BOOL		BuiltInIO://cpu/#0/Input_Bit_03	False	False	Publish Only	Motor thermal switch
I_Prot_ByPass	BOOL		BuiltInIO://cpu/#0/Input_Bit_04	False	False	Publish Only	Guards-bypassed selector switch - Under Panel
I_Hub_Position	BOOL		BuiltInIO://cpu/#0/Input_Bit_05	False	False	Publish Only	Hub/chuck in extraction position
I_FrontGuard_Closed	BOOL		BuiltInIO://cpu/#0/Input_Bit_06	False	False	Publish Only	Front guard closed
I_LowSpd_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_07	False	False	Publish Only	Slow speed button
I_VFD_Alarm	BOOL		BuiltInIO://cpu/#0/Input_Bit_08	False	False	Publish Only	Inverter in alarm
I_SafeGuard_Closed	BOOL			False	False	Publish Only	Side doors closed

I_OverRider_PH	BOOL		BuiltInIO://cpu/#0/Input_Bit_13	False	False	Publish Only	Roll diameter photocell
I_OR_Position_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN01 (Pin 1.1) _6000_01	False	False	Publish Only	OverRider position sens
I_OR_Pressure_Sens	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN02 (Pin 2.1) _6000_02	False	False	Publish Only	OverRider pressure switch
I_OR_Auto_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN03 (Pin 1.2) _6000_03	False	False	Publish Only	OverRider in automatic
I_OR_Cont_OS	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN04 (Pin 2.2) _6000_04	False	False	Publish Only	OverRider in continuous
O_MotorStart	BOOL		BuiltInIO://cpu/#0/Output_Bit_00	False	False	Publish Only	Motor start
I_FanTemp_Ok	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN06 (Pin 2.3) _6000_06	False	False	Publish Only	Motor (fan) thermal OK
O_Alarm_Array	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT01 (Pin 1.1) _7000_01	False	False	Publish Only	Alarm lamp
O_Prot_ByPass_Lamp	BOOL			False	False	Publish Only	Guards bypassed lamp
O_Motor_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_01	False	False	Publish Only	Motor ON lamp
O_RwUnload	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-ECO Output_OUT03 (Pin 1.2) _7000_03	False	False	Publish Only	Unloaders enable - opening
O_FrontGuard_ByPass	BOOL		ECAT://node#[1,1]/IB IL 24 DO 8/HD-	False	False	Publish Only	Front guard bypass

			ECO Output_OUT 04 (Pin 2.2) _7000_04				
O_Brake	BOOL		BuiltInIO://cpu/#0/Output_Bit_02	False	False	Publish Only	Brake command
O_OR_Auto_Lamp	BOOL		BuiltInIO://cpu/#0/Output_Bit_06	False	False	Publish Only	OverRider auto/continuous lamp
O_OverRider_Up	BOOL		BuiltInIO://cpu/#0/Output_Bit_09	False	False	Publish Only	Raise OverRider ~10mm Over Rewinder roll
O_OverRider_Hold	BOOL		BuiltInIO://cpu/#0/Output_Bit_04	False	False	Publish Only	OverRider Lock
O_CoolingFan	BOOL		BuiltInIO://cpu/#0/Output_Bit_03	False	False	Publish Only	Brake and motor fans
Fan_Timeout	REAL	0.5		True	False	Publish Only	Fans stop timeout [min]
OverRider_TimeOFF	REAL	0.01		True	False	Publish Only	OverRider photocell OFF timeout (panel)
OverRider_TimeON	REAL	0.05		True	False	Publish Only	OverRider photocell ON timeout (panel)
AlarmOn	BOOL			False	False	Publish Only	Alarm active
BuzzerOn	BOOL			False	False	Publish Only	Panel buzzer command
Msg	INT			False	False	Publish Only	Message
Index_B	INT			False	False	Publish Only	Alarm index
MotStop	BOOL			False	False	Publish Only	Motor stopped
Run_Normal_Spd	BOOL			False	False	Publish Only	Motor running - normal speed
Clock1s	BOOL			False	False	Publish Only	1 s clock
ManMode	BOOL			True	False	Publish Only	Manual mode
Speed_Man_SP	INT			True	False	Publish Only	% speed in manual
Counter_Reset	BOOL			False	False	Publish Only	Counter reset from panel
I_FrontGuard_Open	BOOL		BuiltInIO://cpu/#0/Input_Bit_09	False	False	Publish Only	Front guard open
RwUnload_Enable	BOOL		BuiltInIO://cpu/#0/Output_Bit_07	False	False	Publish Only	Unloaders enable - closing
JogSpd_SP	REAL	10.0		True	False	Publish Only	Speed in manual
O_AccDec2	BOOL		BuiltInIO://cpu/#0/Output_Bit_05	False	False	Publish Only	Second acc/dec ramp
AccelTrim	REAL			True	False	Publish Only	Acceleration coefficient in run
DecelTrim	REAL			True	False	Publish Only	Deceleration coefficient in run
DecelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests decel)
AccelCoarse	REAL			True	False	Publish Only	Acceleration coefficient at start (orig. comment; name suggests accel)
O_AcousticSignal	BOOL		BuiltInIO://cpu/#0/Output_Bit_08	False	False	Publish Only	Horn / Acoustic signal
Pot2_Slow_V	INT		BuiltInIO://o	False	False	Publish Only	Potentiometer 2 value - Fina

elocity			pt/#0/Ch2_Analog_Input_Value				l speed -
I_AutoStop_OS	BOOL		BuiltInIO://cpu/#0/Input_Bit_10	False	False	Publish Only	Stop on metre counter
I_RollEnd_SlowDown	BOOL		BuiltInIO://cpu/#0/Input_Bit_11	False	False	Publish Only	Slow down at end of roll
I_WebBreak_Monitor	BOOL		BuiltInIO://cpu/#0/Input_Bit_12	False	False	Publish Only	Paper break monitoring ON
I_Vacuum_OK	BOOL		IOBus://unit#3/Ch1 Reset/External Input Status/Ch1 External Input 0 Status	False	False	Publish Only	Extractor fan thermal OK
I_SlowSens_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN05 (Pin 1.3) _6000_05	False	False	Publish Only	Slow-down photocell
I_WebBreak_PH	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN07 (Pin 1.4) _6000_07	False	False	Publish Only	Paper break photocell
UwStop_At_Meter	REAL	5100		True	False	Publish Only	Unwinder Stop at metres
UwSlowMargin_m	REAL	10		True	False	Publish Only	Unwinder Slow-down metres
LowSpd_Display	REAL			True	False	Publish Only	Slow-down speed (displayed)
RwMaxDiam	REAL	1340.0		True	False	Publish Only	Rewinder max diameter protection (mm)
RwMaxStop	BOOL			False	False	Publish Only	Rewinder max diameter STOP (latched)
Encoder_Pulse	INT	1000		True	False	Publish Only	Encoder pulses/Rotation
RwSlowDecelLrn	REAL	0.0		True	False	Publish Only	Learned real slow-down decel (mm/s2) - 0 = not learned
LowSpd_Man_SP	INT			False	False	Publish Only	% slow-down speed in manual
CountReset_Req	BOOL			False	False	Publish Only	Counter reset requested
Psw	INT			False	False	Publish Only	Password
Language	DINT			True	False	Publish Only	Language
I_RsetAlarm	BOOL		ECAT://node#[1,0]/IB IL 24 DI 8/HD-ECO Input-_IN08 (Pin 2.4) _6000_08	False	False	Publish Only	Alarm reset

Fix10Vdc_1	UINT			False	False	Publish Only	Fixed 10 Vdc generator
EncoderWheel_Diam	REAL	400		True	False	Publish Only	Encoder wheel diameter
AlarmCodeSV	ARRAY[0..32] OF BOOL			False	False	Publish Only	Panel alarms
SafeGuard_ByPass	BOOL			False	False	Publish Only	Guards bypassed
FrontGuard_Closed	BOOL			False	False	Publish Only	Front guard closed
Anal3	UINT			False	False	Publish Only	(?) Analog 3
Anal4	UINT			False	False	Publish Only	(?) Analog 4
SoftStop	BOOL	False		False	False	Publish Only	Soft stop in progress (HMI)
SoftStopEnable	BOOL	True		True	False	Publish Only	STOP button = soft stop (False = immediate as before)
SoftStopEndSpd	REAL	5.0		True	False	Publish Only	Soft stop: run command drops below this speed (m/min)
UwDiam_Measure	REAL			True	False	Publish Only	Measured distance
UwCoreDiameter	REAL	100		True	False	Publish Only	Core Outside diameter [mm]
Core_Measure	BOOL	False		True	False	Publish Only	Stored core value
UwCoreDistance	REAL	785.56		True	False	Publish Only	Sonar distance to empty core. [mm]
AI_Diam_Unwinder	INT		IOBus://unit#2/Ch1 Analog Input Value	False	False	Publish Only	Analog value - UnWinder Diameter
Measure_ON	BOOL	True		True	False	Publish Only	Measurement enabled
UwDiam_Act	REAL			True	False	Publish Only	Measured roll diameter
UwSlowDown_Diam	REAL	200		True	False	Publish Only	Unwinder Slow-down diameter [mm]
UwDiamStop	REAL	110		True	False	Publish Only	Stop diameter
Alarm_Reset	BOOL			False	False	Publish Only	Alarm reset from panel
Psw22	BOOL			False	False	Publish Only	(?) Password 22
Psw44	BOOL			False	False	Publish Only	(?) Password 44
I_Roll_OK	BOOL			False	False	Publish Only	Roll OK / Broken Holder Joint
I_RollEnd_PH	BOOL			False	False	Publish Only	End-of-roll photocell
AI_Diam_Rewinder	INT		IOBus://unit#2/Ch2 Analog Input Value	False	False	Publish Only	Analog value - ReWinder Diameter
RwStopEnable	BOOL	False		True	False	Publish Only	Rewinder diameter stop enabled
RwTargetDiam	REAL	300.0		True	False	Publish Only	Rewinder final diameter (mm)
RwUseSlow	BOOL	True		True	False	Publish Only	Slow down before target
RwSlowMargin_m	REAL	10.0		True	False	Publish Only	Rewinder Metres at slow speed before stop
RwSlowDecel	REAL	0.0		True	False	Publish Only	Rewinder Slow-down decel mm/s2 - 0 = auto from DecStart
RwResetHyst	REAL	100.0		True	False	Publish Only	Reset requests below target - this [mm]
RwDist0V	REAL	0.0		True	False	Publish Only	DT35 distance at 0 V (mm)
RwDist10V	REAL	750.0		True	False	Publish Only	DT35 distance at 10 V (mm)

RwRawMin	INT	10		True	False	Publish Only	Raw below = sensor fault
RwRawMax	INT	3900		True	False	Publish Only	Raw above = sensor fault
RwStopOnFault	BOOL	True		False	False	Publish Only	Stop on sensor fault
RwAxisDist	REAL	675.3		True	False	Publish Only	Sensor to shaft centre (mm) - teach
RwFilter_K	REAL	0.2		True	False	Publish Only	Distance filter per 100 ms (0..1)
RwWinTicks	INT	100		True	False	Publish Only	Material learn window (100 = 10 s)
RwMinPulses	DINT	1000		True	False	Publish Only	Min pulses in window to learn
RwLearn_K	REAL	0.3		True	False	Publish Only	Material factor filter weight
RwMat_K	REAL	0.0		True	False	Publish Only	Learned material factor
RwLearnStop	BOOL	True		True	False	Publish Only	Learn stop decel on every stop
RwMinLearnSpd	REAL	50.0		True	False	Publish Only	Min speed to learn a stop (m/min)
RwStopDecel	REAL	0.0		True	False	Publish Only	Learned stop decel (pulses/s2)
RwTreact	REAL	0.1		True	False	Publish Only	Stop reaction time (s)
RwTstopDef	REAL	3.0		True	False	Publish Only	Stop time until decel learned (s)
RwTrim	REAL	1.0		True	False	Publish Only	Stop length trim (>1 = earlier)
RwDist	REAL			False	False	Publish Only	Filtered distance (mm)
RwDiam	REAL			False	False	Publish Only	Rewinder diameter (mm)
RwSpeed	REAL			False	False	Publish Only	Line speed from encoder (m/min)
RwRemain_m	REAL			False	False	Publish Only	Metres to target diameter
RwStopLen_m	REAL			False	False	Publish Only	Stopping length now (m)
RwSlowLen_m	REAL			False	False	Publish Only	Rewinder Slow-down length [m]
RwStopReq	BOOL			False	False	Publish Only	Rewinder diameter STOP request
RwSlowReq	BOOL			False	False	Publish Only	Rewinder diameter SLOW request
RwSensFault	BOOL			False	False	Publish Only	Rewinder DT35 sensor fault
UwStopEnable	BOOL	False		True	False	Publish Only	Unwnder Auto Stop - Enable
SlowAtDisp	REAL			False	False	Publish Only	HMI: slow-down starts at (m or mm, see SlowAtMode)
SlowAtMode	INT			False	False	Publish Only	HMI: 0=none 1=unwinder [m] 2=rewinder [mm]
SlowAtShow	BOOL			False	False	Publish Only	HMI: show slow-down display
UwSlow_at_Metre	REAL			False	False	Publish Only	Unwinder: slow-down starts at [m]
UwSlowReq	BOOL			False	False	Publish Only	Unwinder metre slow-down request (latched)
RwSlowAtDiam	REAL			False	False	Publish Only	Rewinder slow-down starts at diameter [mm]
RwStopAtDiam	REAL			False	False	Publish Only	Rewinder: stop command at diameter (mm)
RwMaxSlow	BOOL			False	False	Publish Only	Rewinder max diameter SLOW (latched)
UwEmptyHyst	REAL	5.0		True	False	Publish Only	Unwinder Empty Hysteresis

	GSM_Learned_OK	BOOL			False	False	Publish Only	Paper [Grams / Square Meter] FALSE until GSM learned
	RwMaxSlowAtDiam	REAL			False	False	Publish Only	Rewinder max: slow-down starts at diameter (mm)
	RwStopDecelK	REAL	0.0		True	False	Publish Only	Learned stop decel per mm of rewinder diameter (mm/s2 per mm)
	SoftStopDecel	REAL	0.0		True	False	Publish Only	Soft stop decel (mm/s2), 0 = DecelCoarse
	SpdMax_Pending	BOOL	False		False	False	Publish Only	New pot range waiting for the motor to stop
	RwAxisDist_Prev	REAL	675.3		True	False	Publish Only	Rewinder: RwAxisDist before the last TEACH (for UNDO)
	UwCoreDist_Prev	REAL	785.56		True	False	Publish Only	Unwinder: UwCoreDistance before the last TEACH (for UNDO)
	RwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO rewinder TEACH (PLC resets it)
	UwTeachUndo	BOOL	False		False	False	Publish Only	HMI: UNDO unwinder TEACH (PLC resets it)